

TaskGrid+

Erweiterbares Aufgabenverarbeitungssystem in Containern

Kurs	Verteilte Systeme — TIK23
Dozent	Kevin Dallmann
Abgabe	08.06.2026
Stand	Juni 2026

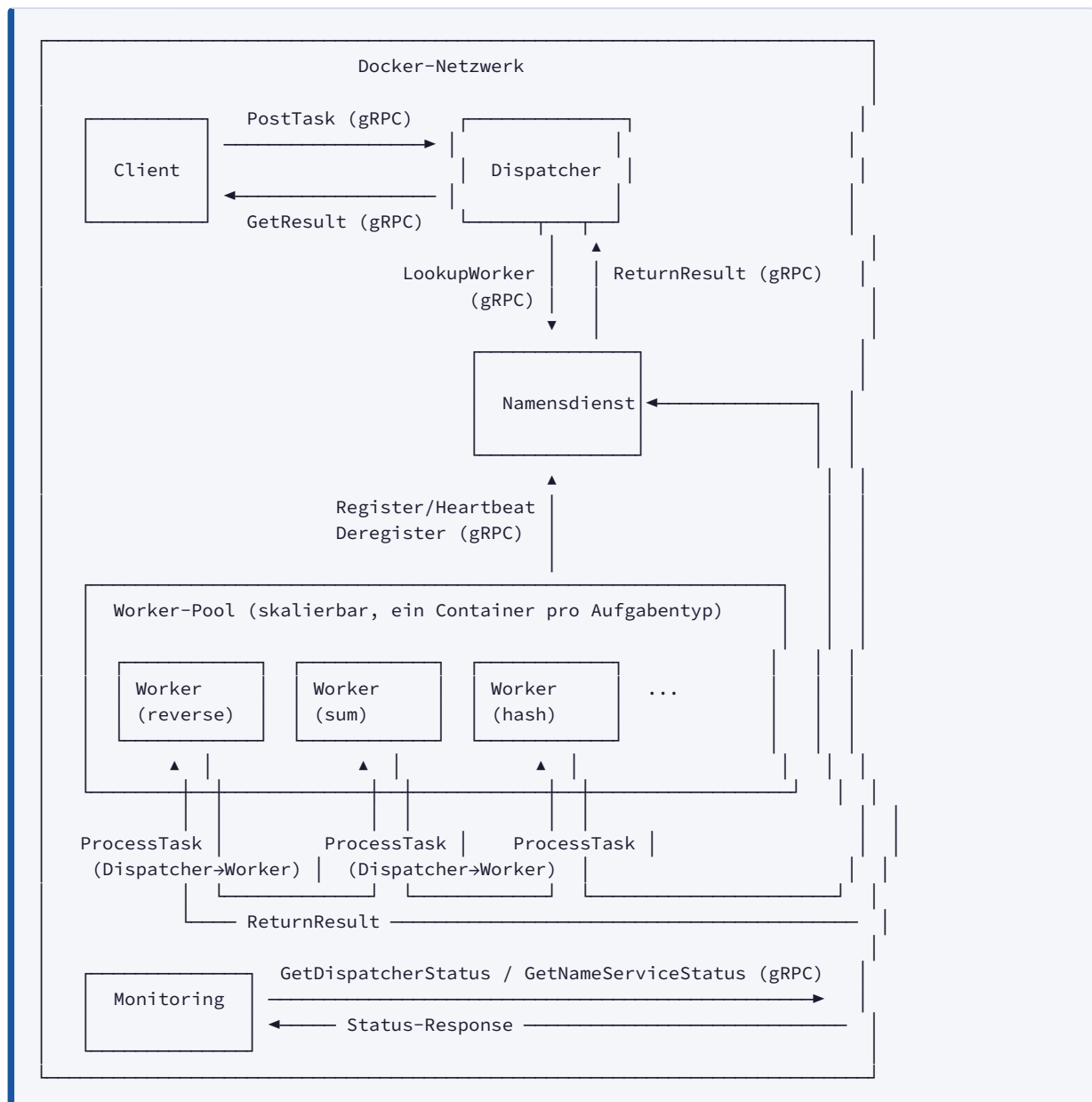
Inhalt

1. Architektur & Komponenten
2. Schnittstellen & Protokoll
3. Task-Zustandsmodell
4. Erweiterbarkeit neuer Tasktypen
5. Startanleitung & Build-Prozess
6. Testprotokoll — Erfolgreiche Tasks
7. Testprotokoll — Fehler- und Robustheitstests
8. Architekturentscheidungen (ADR-001 – ADR-006)

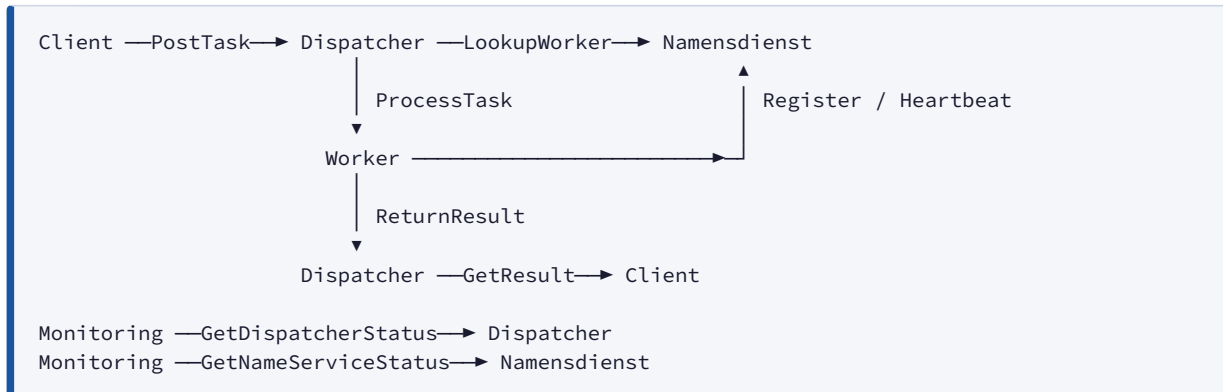
Architekturdokumentation — TaskGrid+

1. Architekturdiagramm

Das folgende Diagramm zeigt alle fünf Komponenten des Systems und ihre Kommunikationsbeziehungen. Alle Verbindungen verwenden gRPC über Protocol Buffers.



Vereinfachte Übersicht der Kommunikationsflüsse



2. Architekturbegründung

Gewählte Komponentenstruktur

Das System ist als lose gekoppelter Verbund eigenständiger Prozesse (Microservices) konzipiert. Jede Komponente läuft in einem eigenen Docker-Container und kommuniziert ausschließlich über gRPC. Diese Entscheidung wurde aus mehreren verteilten-Systeme-Prinzipien abgeleitet:

Verteilungstransparenz (Distribution Transparency)

Aus Sicht des Clients ist es irrelevant, auf welchem Host oder in welchem Container der Dispatcher läuft. Die gRPC-Schnittstelle abstrahiert den physischen Ort vollständig. Dasselbe gilt für den Dispatcher beim Zugriff auf Worker: Über den Namensdienst wird ein Worker per Adresse/Port referenziert — der Dispatcher muss nicht wissen, wie viele Worker-Instanzen existieren oder wo sie physisch laufen.

Fehlertransparenz (Failure Transparency)

Der Dispatcher behandelt Worker-Ausfälle transparent gegenüber dem Client: Überschreitet ein Task den konfigurierten Timeout, wird er automatisch erneut eingeplant (`RETRYING`) und einem anderen Worker zugewiesen. Der Client erhält erst dann eine endgültige Antwort, wenn der Task entweder abgeschlossen (`COMPLETED`) oder definitiv fehlgeschlagen (`FAILED`) ist.

Skalierungstransparenz (Replication Transparency)

Worker können horizontal skaliert werden (`docker compose up --scale worker-sum=3`). Der Namensdienst verwaltet alle registrierten Instanzen, und der Dispatcher wählt per Least-Load-Strategie den am wenigsten ausgelasteten Worker aus. Diese Skalierung ist für den Client vollkommen transparent.

Separation of Concerns

Komponente	Verantwortung
Client	Aufgaben stellen, Ergebnisse abfragen
Dispatcher	Koordination, Queue-Management, Retry-Logik
Namensdienst	Service Discovery, Worker-Gesundheit

Komponente	Verantwortung
Worker	Fachliche Aufgabenverarbeitung
Monitoring	Beobachtbarkeit (Read-only)

Der Dispatcher enthält bewusst keine fachliche Logik — er koordiniert nur. Worker enthalten keine Koordinationslogik — sie verarbeiten nur. Dies ermöglicht das unabhängige Austauschen, Erweitern und Skalieren beider Seiten.

Entkopplung durch Namensdienst

Der Dispatcher kennt Worker-Adressen nicht zur Deployment-Zeit. Stattdessen registrieren sich Worker beim Start beim Namensdienst und kündigen sich bei Shutdown explizit ab. Der Dispatcher fragt den Namensdienst unmittelbar vor der Dispatch-Entscheidung ab (`LookupWorker`). Das System ist dadurch robust gegenüber Worker-Neustarts und dynamischen Skalierungsoperationen.

3. Komponentenbeschreibungen

3.1 Client

Verantwortung:

Stellt Aufgaben an den Dispatcher und fragt Ergebnisse ab. Fungiert als Systemeingang für Endnutzer oder externe Systeme.

Exponierte Schnittstellen: keine (rein aufrufend)

Konsumierte Schnittstellen:

- `DispatcherService.PostTask` — sendet eine neue Aufgabe (Typ + Payload)
- `DispatcherService.GetResult` — fragt den aktuellen Status und das Ergebnis einer Aufgabe per `task_id` ab

Gehaltener Zustand: Die `task_id`, die nach `PostTask` zurückgegeben wird, wird im Prozessspeicher gehalten und für den `GetResult`-Aufruf verwendet. Der Client ist zustandslos über Sitzungen hinaus.

3.2 Dispatcher

Verantwortung:

Zentrale Koordinationskomponente. Nimmt Aufgaben entgegen, verwaltet die Task-Queue, verteilt Aufgaben an geeignete Worker und überwacht deren Bearbeitung (Timeout, Retry).

Exponierte Schnittstellen:

- `DispatcherService.PostTask` — empfängt neue Aufgaben vom Client
- `DispatcherService.GetResult` — liefert Task-Status und -Ergebnis an den Client
- `DispatcherService.ReturnResult` — empfängt Ergebnisse von Workern
- `DispatcherService.GetDispatcherStatus` — liefert Systemmetriken an Monitoring

Konsumierte Schnittstellen:

- `NameService.LookupWorker` — fragt aktive Worker für einen Aufgabentyp ab
- `WorkerService.ProcessTask` — übergibt eine Aufgabe an einen Worker

Gehaltener Zustand:

- `TaskStore` : In-Memory-Dictionary (`task_id` → `TaskRecord`), enthält alle Tasks mit Status, Payload, Ergebnis, Timestamps, Retry-Zähler und zugewiesenem Worker
 - `TaskQueue` : Thread-sichere FIFO-Queue mit `task_id` s wartender Tasks
 - Monitoring-Zähler: Gesamtanzahl Timeouts, Summe Verarbeitungszeiten, Anzahl abgeschlossener Tasks
 - Namensdienst-Stub: gecachter gRPC-Kanal
 - Worker-Channels: Dictionary gecachter gRPC-Kanäle zu Workern (`worker_id` → `grpc.Channel`)
-

3.3 Namensdienst

Verantwortung:

Service-Discovery-Komponente. Verwaltet die Registrierung aktiver Worker, empfängt Heartbeats und meldet inaktive Worker automatisch ab. Ermöglicht dem Dispatcher, passende Worker für einen Aufgabentyp zu finden.

Exponierte Schnittstellen:

- `NameService.RegisterWorker` — Worker registriert sich beim Start
- `NameService.Heartbeat` — Worker meldet sich periodisch mit aktuellem Load
- `NameService.DeregisterWorker` — Worker meldet sich beim Shutdown ab
- `NameService.LookupWorker` — Dispatcher fragt aktive Worker für einen Typ ab
- `NameService.GetNameServiceStatus` — liefert Systemmetriken an Monitoring

Konsumierte Schnittstellen: keine**Gehaltener Zustand:**

- `_registry` : In-Memory-Dictionary (`worker_id` → `Worker`), enthält für jeden registrierten Worker: `worker_id`, `type`, `address`, `port`, `status` (`ACTIVE` / `UNHEALTHY` / `DRAINING` / `OFFLINE`), `last_heartbeat`, `current_load`
 - Hintergrund-Thread (`health-checker`): prüft sekundlich alle Worker und demoviert sie bei ausbleibendem Heartbeat (`ACTIVE` → `UNHEALTHY` → `OFFLINE`)
-

3.4 Worker

Verantwortung:

Spezialisierte Verarbeitungseinheit für einen bestimmten Aufgabentyp (z. B. `reverse`, `sum`, `hash`). Registriert sich beim Start beim Namensdienst, empfängt Aufgaben vom Dispatcher, verarbeitet sie asynchron und sendet das Ergebnis zurück.

Exponierte Schnittstellen:

- `WorkerService.ProcessTask` — empfängt Aufgaben vom Dispatcher; antwortet sofort mit `accepted=true/false`, verarbeitet die Aufgabe in einem Hintergrund-Thread

Konsumierte Schnittstellen:

- `NameService.RegisterWorker` — beim Start (mit Exponential-Backoff-Retry)
- `NameService.Heartbeat` — periodisch (Standard: alle 10 Sekunden) mit aktuellem Load
- `NameService.DeregisterWorker` — beim Shutdown (SIGTERM/SIGINT)
- `DispatcherService.ReturnResult` — nach Abschluss einer Aufgabe

Gehaltener Zustand:

- `_current_load` : Anzahl aktuell laufender Task-Threads (thread-safe, via Lock)
- `_task_threads` : Dictionary (`task_id` → Thread) laufender Verarbeitungs-Threads
- gRPC-Stubs (gecacht): je ein Kanal zum Namensdienst und zum Dispatcher

Aufgabentypen (implementiert):

Typ	Verarbeitung
<code>reverse</code>	Payload-String umkehren
<code>sum</code>	Zahlen (kommagetrennt) summieren
<code>hash</code>	SHA256-Hashwert berechnen
<code>upper</code>	Großschreibung
<code>wait</code>	Verzögerung in Sekunden simulieren
<code>wordcount</code>	Wörter zählen
<code>lower</code>	Kleinschreibung
<code>base64</code>	Base64-Kodierung
<code>prime</code>	Primzahlprüfung

3.5 Monitoring**Verantwortung:**

Reine Beobachtungskomponente. Fragt regelmäßig den Dispatcher und den Namensdienst nach Systemmetriken ab und gibt diese aus. Hat keinen Einfluss auf den Verarbeitungsablauf.

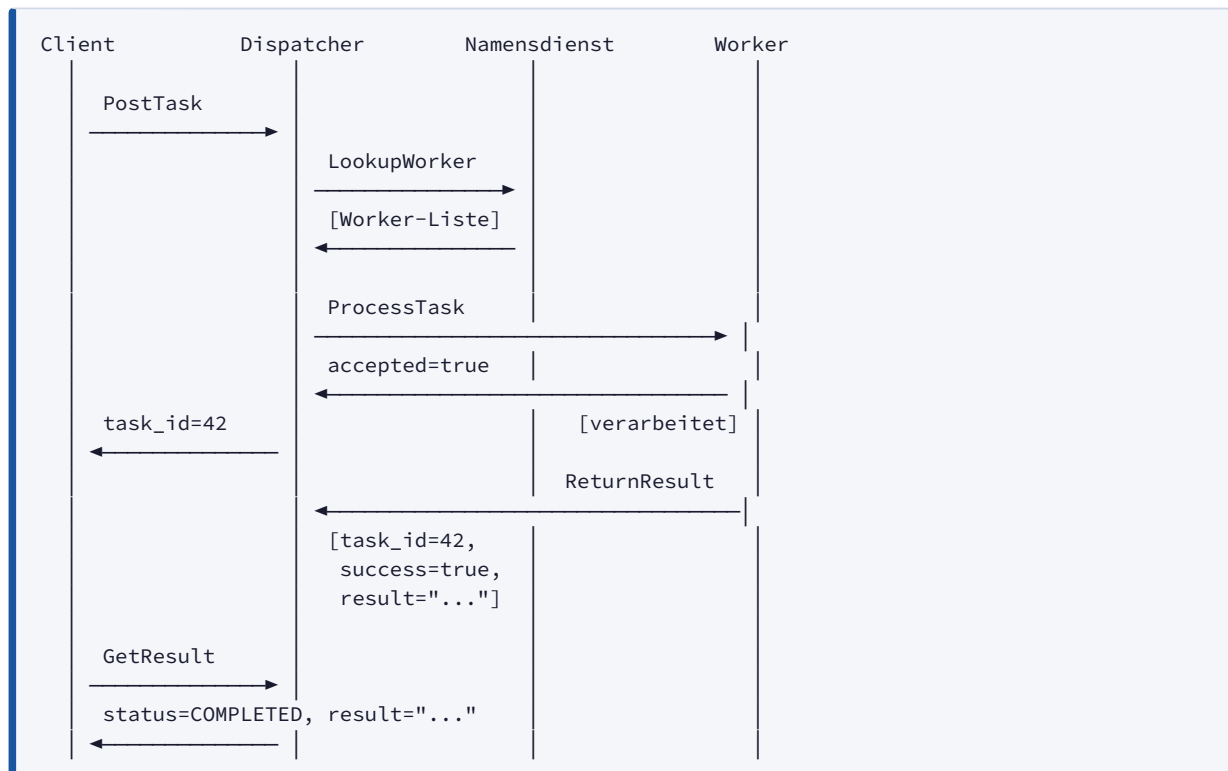
Exponierte Schnittstellen: keine**Konsumierte Schnittstellen:**

- `DispatcherService.GetDispatcherStatus` — Anzahl Tasks (queued, running, completed, failed), Timeouts, Retries, durchschnittliche Verarbeitungszeit
- `NameService.GetNameServiceStatus` — Anzahl registrierter und aktiver Worker, unterstützte Aufgabentypen

Gehaltener Zustand: keiner (zustandslos, poll-basiert)

4. Ablaufdiagramme

4.1 Happy Path: Erfolgreiche Aufgabenverarbeitung

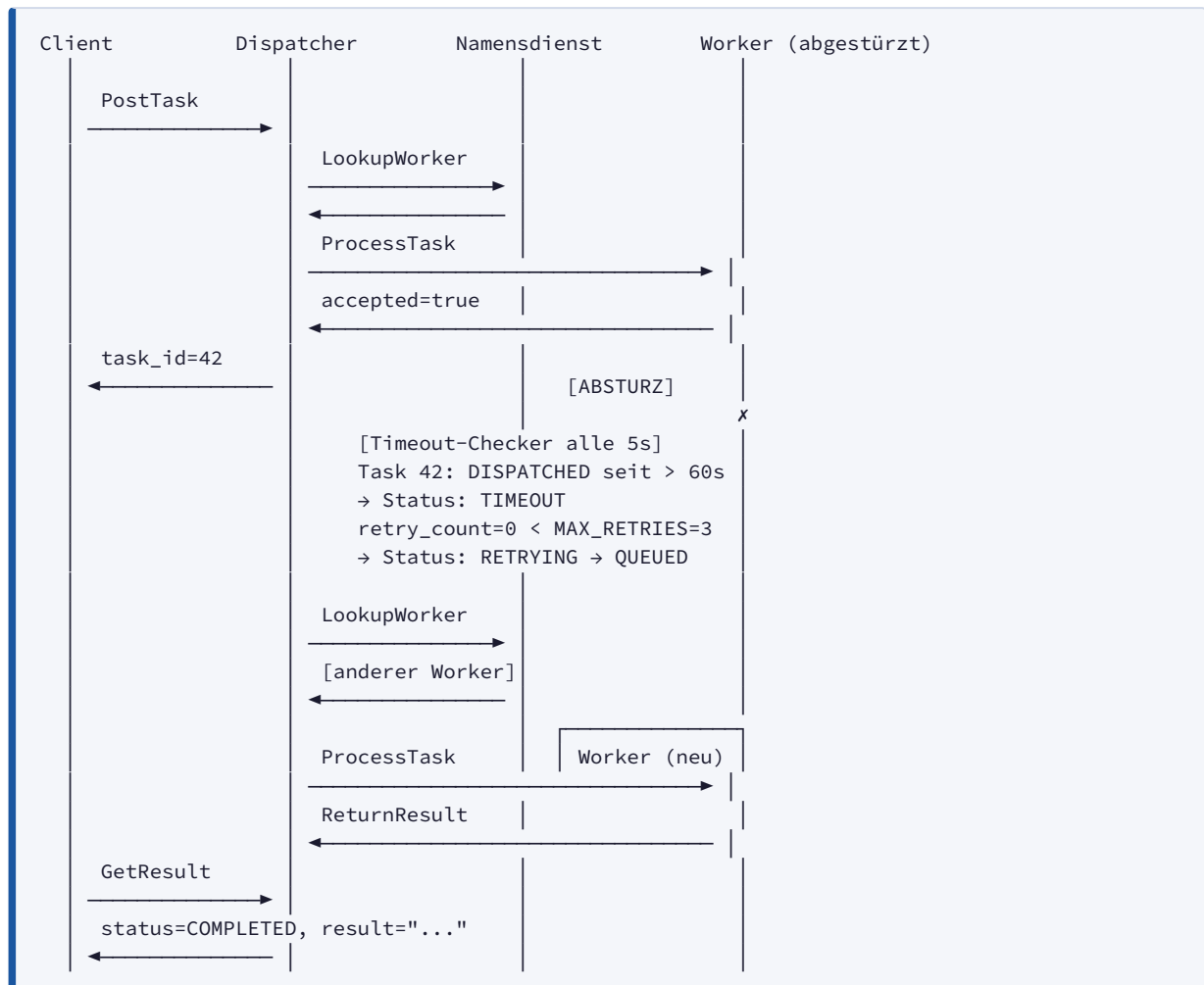


Zustandsübergänge (Task):

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

4.2 Fehlerszenarien

Szenario A: Worker-Absturz / Timeout



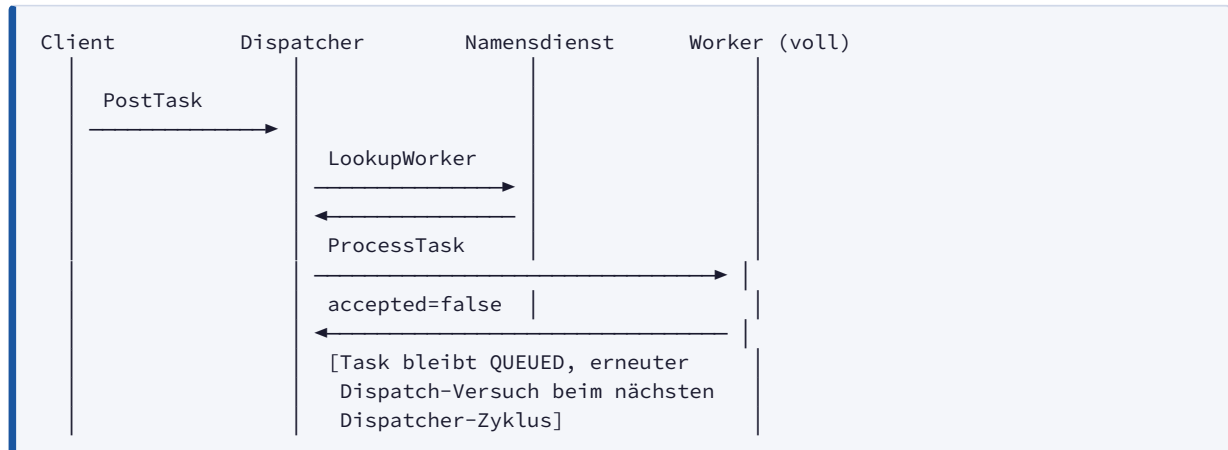
Zustandsübergänge:

DISPATCHED → TIMEOUT → RETRYING → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Bei Erschöpfung aller Retries:

TIMEOUT → RETRYING → QUEUED → ... → TIMEOUT → FAILED

Szenario B: Worker lehnt Aufgabe ab (Kapazität voll)



Szenario C: Unbekannter Aufgabentyp



Ergebnis: Task wird nicht erstellt; Client erhält sofort eine Fehlermeldung.

Schnittstellen- und Protokolldokumentation — TaskGrid+

1. Kommunikationsprotokoll

Warum gRPC?

TaskGrid+ verwendet **gRPC über HTTP/2** als einziges Kommunikationsprotokoll zwischen allen Komponenten. Die Entscheidung ist in [ADR-001](#) ausführlich begründet; zusammengefasst:

Kriterium	gRPC	Rohes UDP
Zustellgarantie	Ja (TCP-basiert)	Nein (Paketverlust möglich)
Typsicherheit	Ja (Protobuf-Schema)	Nein (manuell)
Code-Generierung	Ja (Stubs aus <code>.proto</code>)	Nein
Schnittstellendokumentation	Im Schema eingebettet	Manuell
Streaming	Bidirektional möglich	Manuell

Zuverlässigkeit und Fehlerverhalten

- **Verbindungsaufbau:** gRPC-Kanäle werden lazy initialisiert und gecacht (Worker-Kanäle im Dispatcher, Namensdienst-Stub im Dispatcher und Worker).
- **Timeout-Handling:** Alle kritischen RPC-Aufrufe haben explizite Timeouts (5–10 s). Schlägt ein Aufruf fehl, wird ein `grpc.RpcError` ausgelöst.
- **Retry-Verhalten:** Der Dispatcher re-enqueues Tasks bei nicht erreichbaren Workern (`grpc.RpcError`) oder Ablehnung (`accepted=false`). Details in [ADR-006](#).
- **Fehlerweiterleitung:** Fehler werden nicht als Exceptions an den Client weitergereicht, sondern als strukturierte Antwortfelder (`success=false`, `message`).

MessageHeader (Nachrichtenumschlag)

Jede Anfrage und Antwort enthält einen `MessageHeader` für Tracing und Korrelation:

```
message MessageHeader {
  string message_type = 1; // Name des RPC-Typs
  string request_id    = 2; // UUID für End-to-End-Tracing
  int64  timestamp     = 3; // Unix-Epoch in Millisekunden
  string sender        = 4; // Komponentename (z.B. "dispatcher")
}
```

Beispiel (JSON-Darstellung):

```
{
  "message_type": "PostTaskRequest",
  "request_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "timestamp": 1717200000000,
  "sender": "client"
}
```

2. Datenstrukturen

Task

```
message Task {
    int32    id                = 1; // Eindeutige Task-ID (vom Dispatcher vergeben)
    string   type              = 2; // Aufgabentyp, z.B. "reverse", "sum"
    string   payload           = 3; // Eingabedaten als String
    string   result            = 4; // Ergebnis nach Verarbeitung
    TaskStatus status         = 5; // Aktueller Zustand
    int64    timestamp_created = 6; // Unix-Epoch ms: Zeitpunkt der Erstellung
    int64    timestamp_dispatched = 7; // Unix-Epoch ms: Zeitpunkt des Dispatches
    int64    timestamp_completed = 8; // Unix-Epoch ms: Zeitpunkt des Abschlusses
    int32    retry_count       = 9; // Anzahl bisheriger Retry-Versuche
    string   assigned_worker   = 10; // worker_id des aktuell zugewiesenen Workers
}
```

Worker

```
message Worker {
    string   worker_id        = 1; // Eindeutige Worker-ID
    string   type              = 2; // Aufgabentyp, den dieser Worker verarbeitet
    string   address           = 3; // IP-Adresse / Hostname des Workers
    int32    port              = 4; // gRPC-Port des Workers
    WorkerStatus status       = 5; // Aktueller Gesundheitszustand
    int64    last_heartbeat    = 6; // Unix-Epoch ms: letzter empfangener Heartbeat
    int32    current_load      = 7; // Anzahl aktuell verarbeiteter Tasks
}
```

TaskStatus-Enum

Wert	Name	Bedeutung
0	TASK_STATUS_UNSPECIFIED	Ungültig / unbekannte Task-ID
1	CREATED	Validiert, noch nicht in Queue
2	QUEUED	Wartet in der Dispatcher-Queue
3	DISPATCHED	An Worker gesendet, warte auf Bestätigung
4	PROCESSING	Worker hat angenommen, verarbeitet
5	COMPLETED	Erfolgreich abgeschlossen
6	FAILED	Endgültig fehlgeschlagen
7	TIMEOUT	Kein Ergebnis innerhalb TASK_TIMEOUT_SEC
8	RETRYING	Wird erneut eingeplant

WorkerStatus-Enum

Wert	Name	Bedeutung
0	WORKER_STATUS_UNSPECIFIED	Ungültig
1	ACTIVE	Erreichbar, nimmt Tasks an
2	UNHEALTHY	Kein Heartbeat seit ≥ 30 s — nicht für Dispatch verfügbar
3	DRAINING	Geordneter Shutdown läuft
4	OFFLINE	Kein Heartbeat seit ≥ 90 s — gilt als ausgefallen

3. RPC-Dokumentation

3.1 DispatcherService (Port 50051)

PostTask — Client → Dispatcher

Stellt einen neuen Task in die Queue.

Request:

```
| Feld | Typ | Beschreibung |
|-----|-----|-----|
| header | MessageHeader | Korrelations-ID, Absender |
| type | string | Aufgabentyp (z.B. "reverse" ) — darf nicht leer sein |
| payload | string | Eingabedaten — darf nicht leer sein |
```

Response:

```
| Feld | Typ | Beschreibung |
|-----|-----|-----|
| header | MessageHeader | |
| success | bool | true = Task eingereicht |
| task_id | int32 | Vergebene Task-ID (nur wenn success=true ) |
| message | string | Fehlerbeschreibung (nur wenn success=false ) |
```

Fehlerfälle:

- type oder payload leer → success=false
- Kein Worker für den angegebenen Typ registriert → success=false

GetResult — Client → Dispatcher

Frägt Status und Ergebnis eines Tasks ab.

Request:

```
| Feld | Typ | Beschreibung |
|-----|-----|-----|
```

header	MessageHeader	
task_id	int32	ID des abzufragenden Tasks

Response:

Feld	Typ	Beschreibung
-----	-----	-----
header	MessageHeader	
success	bool	false wenn Task-ID unbekannt
task_id	int32	
result	string	Ergebnis (nur wenn COMPLETED)
status	TaskStatus	Aktueller Task-Zustand
message	string	Fehlerbeschreibung (bei FAILED / TIMEOUT)

Fehlerfälle:

- Unbekannte task_id → success=false, status=TASK_STATUS_UNSPECIFIED

ReturnResult — Worker → Dispatcher

Worker liefert das Verarbeitungsergebnis zurück.

Request:

Feld	Typ	Beschreibung
-----	-----	-----
header	MessageHeader	
task_id	int32	ID des abgeschlossenen Tasks
result	string	Ergebnis-Payload
worker_id	string	ID des Senders
success	bool	false wenn Verarbeitungsfehler aufgetreten
error_message	string	Fehlerbeschreibung (nur wenn success=false)

Response:

Feld	Typ	Beschreibung
-----	-----	-----
header	MessageHeader	
success	bool	
message	string	Fehlerbeschreibung

Fehlerfälle:

- Unbekannte task_id → abgelehnt
- Task nicht in Zustand DISPATCHED / PROCESSING (z.B. bereits COMPLETED) → abgelehnt (Duplikat-Schutz)
- worker_id stimmt nicht mit zugewiesenem Worker überein → abgelehnt

GetDispatcherStatus — Monitoring → Dispatcher

Liefert aktuelle Systemmetriken des Dispatchers.

Request: nur `header`

Response:

Feld	Typ	Beschreibung
<code>workers_registered</code>	int32	Bekannte Worker-Adressen im Channel-Cache
<code>workers_active</code>	int32	Aktive Worker laut Namensdienst
<code>supported_task_types</code>	repeated string	Verfügbare Aufgabentypen
<code>tasks_queued</code>	int32	Tasks aktuell in Queue
<code>tasks_running</code>	int32	Tasks in <code>DISPATCHED</code> / <code>PROCESSING</code>
<code>tasks_completed</code>	int32	Abgeschlossene Tasks
<code>tasks_failed</code>	int32	Fehlgeschlagene Tasks
<code>avg_processing_time_ms</code>	double	Ø Verarbeitungsdauer in ms
<code>total_timeouts</code>	int32	Gesamtanzahl Timeouts
<code>total_retries</code>	int32	Gesamtanzahl Retries

3.2 NameService (Port 50052)

`RegisterWorker` — Worker → Namensdienst

Worker meldet sich beim Start an.

Request:

Feld	Typ	Beschreibung
<code>header</code>	MessageHeader	
<code>worker_id</code>	string	Eindeutige Worker-ID
<code>type</code>	string	Aufgabentyp des Workers
<code>address</code>	string	Erreichbare IP / Hostname
<code>port</code>	int32	gRPC-Port des Workers
<code>capacity</code>	int32	Maximale parallele Tasks (optional)

Response: `success`, `message`

Verhalten: Worker versucht die Registrierung mit Exponential-Backoff-Retry (bis zu 10 Versuche, max. 30 s Wartezeit).

`Heartbeat` — Worker → Namensdienst

Worker signalisiert Verfügbarkeit periodisch (Standard: alle 10 s).

Request:

Feld	Typ	Beschreibung
<code>header</code>	MessageHeader	
<code>worker_id</code>	string	

timestamp	int64	Unix-Epoch ms
current_load	int32	Anzahl aktuell verarbeiteter Tasks

Response: success, message

Fehlerfälle:

- Unbekannter worker_id (z.B. Namensdienst neu gestartet) → success=false; Worker versucht Re-Registrierung.

LookupWorker — Dispatcher → Namensdienst

Dispatcher fragt aktive Worker für einen bestimmten Typ ab.

Request:

Feld	Typ	Beschreibung
-----	-----	-----
header	MessageHeader	
type	string	Gesuchter Aufgabentyp

Response:

Feld	Typ	Beschreibung
-----	-----	-----
success	bool	
workers	repeated Worker	Alle ACTIVE Worker des Typs
message	string	

DeregisterWorker — Worker → Namensdienst

Worker meldet sich beim Shutdown explizit ab.

Request: header, worker_id

Response: success, message

Verhalten: Wird bei SIGTERM/SIGINT ausgelöst; Worker wechselt intern in den Zustand DRAINING.

GetNameServiceStatus — Monitoring → Namensdienst

Response: analog zu GetDispatcherStatus (workers_registered, workers_active, supported_task_types, ...)

3.3 WorkerService (Port 50053 pro Container)

ProcessTask — Dispatcher → Worker

Dispatcher übergibt einen Task zur Verarbeitung.

Request:

Feld	Typ	Beschreibung
------	-----	--------------

```
|-----|-----|-----|
| header | MessageHeader | |
| task   | Task | Vollständiges Task-Objekt inkl. Payload |
```

Response:

```
| Feld | Typ | Beschreibung |
|-----|-----|-----|
| header | MessageHeader | |
| accepted | bool | true = Worker nimmt Task an und verarbeitet ihn im Hintergrund |
| message | string | Ablehnungsgrund (wenn accepted=false) |
```

Verhalten: Der Worker antwortet sofort (nicht-blockierend). Die Verarbeitung läuft in einem Hintergrund-Thread. Das Ergebnis wird anschließend via `ReturnResult` an den Dispatcher gesendet.

Fehlerfälle:

- Worker am Kapazitätslimit (`current_load >= capacity`) → `accepted=false`
- Worker im Shutdown (`stop_event` gesetzt) → `accepted=false`

4. Port-Übersicht

Komponente	Port (intern)	Port (extern/Host)	Konfiguration
Dispatcher	50051	50051	<code>DISPATCHER_PORT</code>
Namensdienst	50052	50052	<code>NAMESERVICE_PORT</code>
Worker (je Instanz)	50053	— (kein Host-Mapping)	<code>WORKER_PORT</code>
Monitoring	—	—	(nur ausgehend)
Client	—	—	(nur ausgehend)

Worker sind nur innerhalb des Docker-Netzwerks erreichbar — kein direkter Zugriff von außen vorgesehen.

5. Erweiterbarkeit — Neuen Aufgabentyp hinzufügen

Dank der Trennung zwischen Dispatcher (Koordination) und Worker (Fachlogik) sind **keine Änderungen am Dispatcher, Namensdienst oder Client** erforderlich, um einen neuen Aufgabentyp zu ergänzen.

Schritt 1: Handler implementieren

In `worker/handlers.py` eine neue Funktion hinzufügen:


```
def handle_mytype(payload: str) -> tuple[bool, str]:
    try:
        result = do_something(payload)
        return True, result
    except Exception as e:
        return False, f"Error: {str(e)}"
```

Schritt 2: Handler registrieren

```
HANDLERS = {
    "reverse": handle_reverse,
    "sum":      handle_sum,
    # ...
    "mytype":  handle_mytype, # ← hier eintragen
}
```

Schritt 3: Worker-Container starten

```
# Direkt:
WORKER_ID=worker-mytype-1 WORKER_TYPE=mytype python worker/main.py

# Via Docker Compose:
docker compose up --scale worker-mytype=2
```

Was beim Start passiert

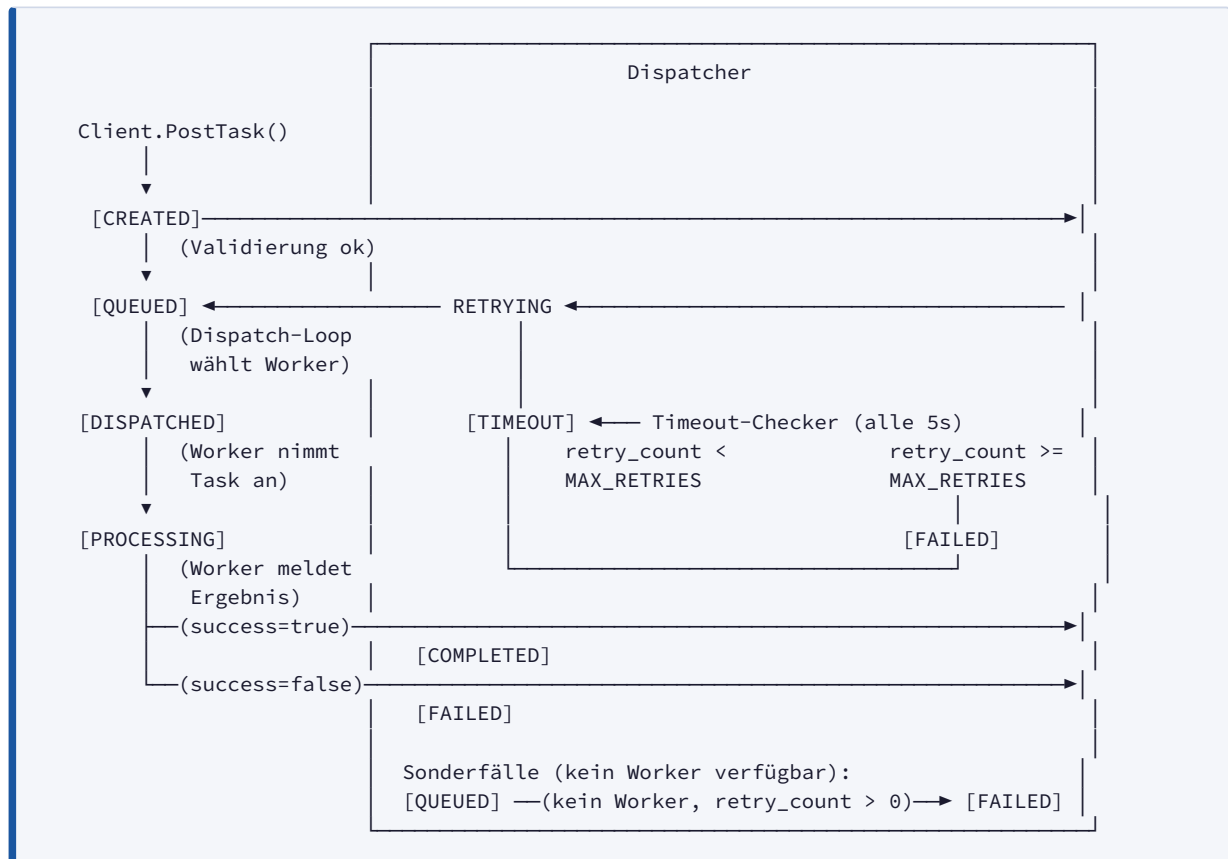
1. Worker liest `WORKER_TYPE=mytype` aus der Umgebung.
2. Worker sendet `RegisterWorker(type="mytype", ...)` an den Namensdienst.
3. Namensdienst speichert den Worker in seiner Registry.
4. Dispatcher findet den neuen Worker beim nächsten `LookupWorker(type="mytype")`.
5. Client kann sofort `PostTask(type="mytype", payload="...")` senden.

Welche Komponenten müssen geändert werden?

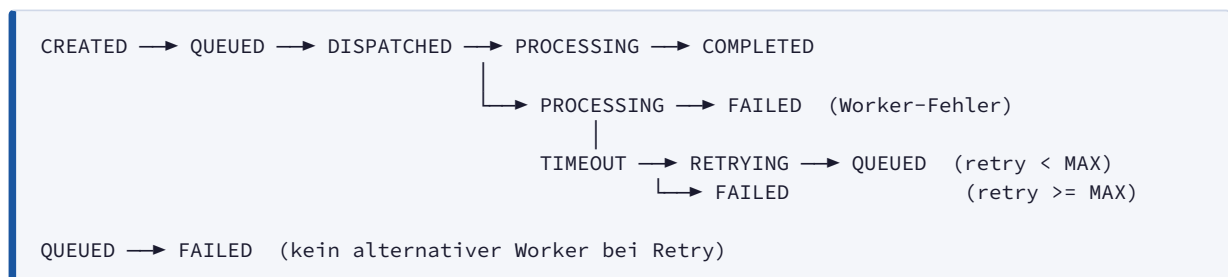
Komponente	Änderung nötig?
<code>worker/handlers.py</code>	Ja — Handler hinzufügen
<code>docker-compose.yml</code>	Optional — neuen Worker-Service eintragen
Dispatcher	Nein
Namensdienst	Nein
Client	Nein
Proto-Schema	Nein

Task-Zustandsmodell – TaskGrid+

1. Zustandsdiagramm



Vereinfachte Zustandsübergänge



2. Beschreibung aller 8 Zustände

#	Status	Beschreibung
0	<code>TASK_STATUS_UNSPECIFIED</code>	Ungültiger/unbekannter Status — wird nur beim Lookup einer unbekannten <code>task_id</code> zurückgegeben
1	<code>CREATED</code>	

#	Status	Beschreibung
		Task wurde validiert und vom Dispatcher entgegengenommen; wird unmittelbar in <code>QUEUED</code> überführt (kein persistenter Zwischenzustand in der Queue)
2	<code>QUEUED</code>	Task wartet in der In-Memory-FIFO-Queue auf Dispatch; auch Zielzustand nach einem Retry
3	<code>DISPATCHED</code>	Dispatcher hat dem Worker eine <code>ProcessTask</code> -Anfrage geschickt; wartet auf Bestätigung (<code>accepted</code>)
4	<code>PROCESSING</code>	Worker hat den Task angenommen (<code>accepted=true</code>); Verarbeitung läuft im Worker-Thread
5	<code>COMPLETED</code>	Worker hat <code>ReturnResult</code> mit <code>success=true</code> zurückgesendet; Ergebnis ist abrufbar
6	<code>FAILED</code>	Task endgültig fehlgeschlagen — entweder Worker-Fehler (<code>success=false</code>), Timeout nach Ausschöpfung aller Retries, kein alternativer Worker verfügbar oder ungültiger Typ/Payload
7	<code>TIMEOUT</code>	Dispatcher hat keinen <code>ReturnResult</code> -Aufruf innerhalb von <code>TASK_TIMEOUT_SEC</code> erhalten; kurzlebiger Zwischenzustand vor <code>RETRYING</code> oder <code>FAILED</code>
8	<code>RETRYING</code>	Task wird erneut eingeplant; kurzlebiger Zwischenzustand — <code>retry_count</code> wird inkrementiert, <code>last_failed_worker</code> gesetzt, danach sofort <code>QUEUED</code>

3. Übergangstabelle (State × Event → Neuer Status)

Ausgangszustand	Ereignis / Bedingung	Auslösende Komponente	Neuer Status
—	<code>PostTask</code> empfangen, Validierung ok	Dispatcher (<code>post_task</code>)	<code>CREATED</code>
<code>CREATED</code>	Task in Store gespeichert und in Queue eingereiht	Dispatcher (<code>post_task</code>)	<code>QUEUED</code>
<code>QUEUED</code>	Dispatch-Loop wählt Worker aus, sendet <code>ProcessTask</code>	Dispatcher (<code>_dispatch_to_worker</code>)	<code>DISPATCHED</code>
<code>QUEUED</code>	Kein Worker verfügbar, <code>retry_count > 0</code> (Retry ohne Alternative)	Dispatcher (<code>_dispatch_loop</code>)	<code>FAILED</code>
<code>QUEUED</code>	Kein Worker verfügbar, <code>retry_count == 0</code> (Erstversuch)	Dispatcher (<code>_dispatch_loop</code>)	<code>QUEUED</code> (re-enqueue, kurzes Backoff)

Ausgangszustand	Ereignis / Bedingung	Auslösende Komponente	Neuer Status
DISPATCHED	Worker antwortet mit <code>accepted=true</code>	Dispatcher (<code>_dispatch_to_worker</code>)	PROCESSING
DISPATCHED	Worker antwortet mit <code>accepted=false</code> (Kapazität voll)	Dispatcher (<code>_dispatch_to_worker</code>)	QUEUED (re-enqueue)
DISPATCHED	Worker nicht erreichbar (gRPC-Fehler)	Dispatcher (<code>_dispatch_to_worker</code>)	QUEUED (re-enqueue)
DISPATCHED	Elapsed > <code>TASK_TIMEOUT_SEC</code>	Dispatcher (<code>_timeout_loop</code> , alle 5s)	TIMEOUT
PROCESSING	Worker sendet <code>ReturnResult</code> mit <code>success=true</code>	Dispatcher (<code>return_result</code>)	COMPLETED
PROCESSING	Worker sendet <code>ReturnResult</code> mit <code>success=false</code>	Dispatcher (<code>return_result</code>)	FAILED
PROCESSING	Elapsed > <code>TASK_TIMEOUT_SEC</code>	Dispatcher (<code>_timeout_loop</code> , alle 5s)	TIMEOUT
TIMEOUT	<code>retry_count</code> < <code>MAX_RETRIES</code>	Dispatcher (<code>_timeout_loop</code>)	RETRYING
TIMEOUT	<code>retry_count</code> >= <code>MAX_RETRIES</code>	Dispatcher (<code>_timeout_loop</code>)	FAILED
RETRYING	<code>retry_count</code> inkrementiert, <code>last_failed_worker</code> gesetzt	Dispatcher (<code>_timeout_loop</code>)	QUEUED
COMPLETED	—	—	(Terminalzustand)
FAILED	—	—	(Terminalzustand)

4. Verhinderung ungültiger Zustandsübergänge

Der Dispatcher implementiert explizite Guards, die ungültige Zustandsübergänge abweisen:

Doppelte Ergebnisse (Duplicate Results)

`ReturnResult` wird nur verarbeitet, wenn der Task im Zustand `DISPATCHED` oder `PROCESSING` ist:

```

if task.status not in (taskgrid_pb2.DISPACHED, taskgrid_pb2.PROCESSING):
    if task.status == taskgrid_pb2.COMPLETED:
        # Doppeltes Ergebnis – ignorieren, Warnung loggen
        return ReturnResultResponse(success=False, message="task already completed")
    else:
        # Veralteter Zustand (z. B. FAILED nach Timeout) – ignorieren
        return ReturnResultResponse(success=False, message="task not in active state")

```

Praktischer Fall: Worker verarbeitet einen Task, der zwischenzeitlich vom Timeout-Checker auf `FAILED` gesetzt wurde. Das verspätete `ReturnResult` des Workers wird stillschweigend verworfen.

Falsche Worker-Zuordnung (Wrong Worker)

Wenn ein `ReturnResult` von einem anderen Worker als dem zugewiesenen eintrifft, wird es abgelehnt:

```

if task.assigned_worker and worker_id != task.assigned_worker:
    return ReturnResultResponse(success=False, message="task assigned to different worker")

```

Validierung vor Task-Erstellung

Bei `PostTask` wird geprüft, ob `type` und `payload` nicht leer sind. Ist kein Worker für den angegebenen Typ beim Namensdienst registriert, wird der Task gar nicht erst erstellt (`success=false` , kein Zustandsübergang).

5. Timeout- und Retry-Verhalten

Timeout-Erkennung

Ein Hintergrund-Thread (`timeout-checker`) prüft alle **5 Sekunden** alle Tasks im Zustand `DISPACHED` oder `PROCESSING` :

```

elapsed_ms = now_ms - task.timestamp_dispatched
if elapsed_ms > TASK_TIMEOUT_SEC * 1000:
    → TIMEOUT

```

Konfiguration via Umgebungsvariable `TASK_TIMEOUT_SEC` (Standard: **60 Sekunden**).

Retry-Ablauf

```

TIMEOUT erkannt
├── retry_count < MAX_RETRIES
│   ├── retry_count += 1
│   ├── last_failed_worker = timed_out_worker
│   ├── assigned_worker = ""
│   ├── timestamp_dispatched = 0
│   ├── Status: RETRYING
│   └── Status: QUEUED → Task in Queue einreihen
└── retry_count >= MAX_RETRIES
    └── Status: FAILED
        error_message = "task timed out after maximum retries"
  
```

Konfiguration via Umgebungsvariable `MAX_RETRIES` (Standard: **3**).

Worker-Ausschluss bei Retry

Der Dispatcher schließt bei der Worker-Auswahl nach einem Timeout den zuletzt fehlgeschlagenen Worker aus (`last_failed_worker`). Falls nur ein Worker verfügbar ist, fällt er auf diesen zurück. Falls kein alternativer Worker gefunden wird und `retry_count > 0` , wird der Task sofort auf `FAILED` gesetzt.

Neueinplanung (Re-Scheduling)

Tasks werden in folgenden Fällen zurück in die Queue gestellt (ohne Retry-Zähler zu erhöhen):

Ursache	Bedingung
Worker lehnt ab (<code>accepted=false</code>)	Kapazitätsgrenze erreicht
Worker nicht erreichbar (gRPC-Fehler)	Netzwerkfehler beim <code>ProcessTask</code> -Aufruf
Namensdienst temporär nicht verfügbar	<code>LookupWorker</code> -RPC schlägt fehl
Kein Worker registriert (Erstversuch)	<code>retry_count == 0</code> , kurzes Backoff (1s)

Adding New Task Types

This guide explains how to add a new task type handler to the Worker without modifying the Dispatcher.

Architecture

TaskGrid uses a **data-driven handler registration system**:

1. All task handlers are registered in a HANDLERS dictionary in `worker/handlers.py`
2. Each worker declares its supported task type via the `WORKER_TYPE` environment variable at startup
3. The Namensdienst learns the type via the `RegisterWorkerRequest`
4. The Dispatcher discovers workers for a task type via `LookupWorker` to the Namensdienst
5. No Dispatcher code changes needed when adding new types

Adding a New Handler

Step 1: Define the Handler Function

Add a new handler function to `worker/handlers.py` :

```
def handle_mytype(payload: str) -> tuple[bool, str]:
    """Brief description of what this handler does.

    Example: "input" -> "output"
    """
    try:
        # Your implementation
        result = process(payload)
        return True, result
    except Exception as e:
        return False, f"Error: {str(e)}"
```

Key requirements:

- Function signature: `handle_<typename>(payload: str) -> tuple[bool, str]`
- Returns: `(success: bool, result: str)`
- If `success=True` : `result` is the task result
- If `success=False` : `result` is the error message
- Validate input and handle errors gracefully
- Never raise exceptions (catch and return False with error message)

Step 2: Register the Handler

Add your handler to the `HANDLERS` dictionary in `worker/handlers.py` :

```
HANDLERS = {
    "reverse": handle_reverse,
    "sum": handle_sum,
    "hash": handle_hash,
    "upper": handle_upper,
    "wait": handle_wait,
    "mytype": handle_mytype, # ← Add your handler here
}
```

The key in the dictionary is the task type name that will be used in:

- Client requests: `PostTask(type="mytype", payload="...")`
- Worker registration: `WORKER_TYPE=mytype`
- Dispatcher routing

Step 3: Start a Worker with the New Type

Start a worker container with the new handler:

```
export WORKER_ID=worker-mytype-1
export WORKER_TYPE=mytype
python worker/main.py
```

The worker will:

1. Validate that `mytype` is in the supported types
2. Load the handler
3. Register with Namensdienst as type `mytype`
4. Begin accepting tasks of type `mytype` from Dispatcher

Step 4: Send Tasks

Clients and the Dispatcher can now send tasks of this type:

```
# Client example
client.post_task(type="mytype", payload="input_data")
```


Example: Adding a "multiply" Handler

1. Add handler to `worker/handlers.py` :

```
def handle_multiply(payload: str) -> tuple[bool, str]:
    """Multiply a comma-separated list of numbers.

    Example: "2,3,4" → "24"
    """
    try:
        parts = payload.split(",")
        numbers = []
        for part in parts:
            stripped = part.strip()
            if not stripped:
                continue
            try:
                numbers.append(float(stripped))
            except ValueError:
                return False, f"Invalid number: '{stripped}'"

        if not numbers:
            return False, "No valid numbers found"

        result = 1
        for n in numbers:
            result *= n

        if result == int(result):
            return True, str(int(result))
        else:
            return True, str(result)
    except Exception as e:
        return False, f"Multiply failed: {str(e)}"
```

2. Register in HANDLERS:

```
HANDLERS = {
    "reverse": handle_reverse,
    "sum": handle_sum,
    "hash": handle_hash,
    "upper": handle_upper,
    "wait": handle_wait,
    "multiply": handle_multiply, # ← New handler
}
```

3. Start worker:

```
WORKER_ID=worker-multiply-1 WORKER_TYPE=multiply python worker/main.py
```

4. Send task:

```
client.post_task(type="multiply", payload="2,3,4")
# Result: "24"
```

Testing Your Handler

Test the handler directly before deploying:

```
from worker.handlers import handle_task

success, result = handle_task("mytype", "test_payload")
if success:
    print(f"Result: {result}")
else:
    print(f"Error: {result}")
```

Error Handling Best Practices

1. **Validate input** - Check for empty payloads, invalid formats, etc.
2. **Return meaningful errors** - Include details about what went wrong
3. **Handle edge cases** - Test with empty strings, very large numbers, special characters, etc.
4. **Implement limits** - Cap execution time, memory usage, output size if needed
5. **Be idempotent** - Tasks may be retried; avoid side effects

Example Error Handling:

```
def handle_process(payload: str) -> tuple[bool, str]:
    # Validate input
    if not payload or not payload.strip():
        return False, "Payload cannot be empty"

    # Validate format
    if not payload.isdigit():
        return False, f"Expected digits, got: {payload}"

    # Check limits
    num = int(payload)
    if num > 1_000_000:
        return False, f"Input exceeds maximum: {num} > 1000000"

    # Process
    try:
        result = expensive_operation(num)
        return True, str(result)
    except TimeoutError:
        return False, "Processing timeout"
    except Exception as e:
        return False, f"Processing failed: {str(e)}"
```

Architecture Implications

No Dispatcher Changes Required

1. ✓ Dispatcher doesn't have a hardcoded list of task types
2. ✓ Dispatcher learns available workers via Namensdienst lookup
3. ✓ Worker type is declared at startup via `WORKER_TYPE` env var

4. ✓ Multiple workers can handle the same type (load balancing)
5. ✓ Easy to scale: `docker-compose up --scale worker-mytype=3`

Docker Compose

To scale a new task type in Docker Compose:

```
services:
  worker-mytype:
    image: taskgrid:worker
    environment:
      WORKER_ID: worker-mytype-${COMPOSE_SERVICE_NUMBER}
      WORKER_TYPE: mytype
      NAMESERVICE_ADDRESS: nameservice
      DISPATCHER_ADDRESS: dispatcher
    depends_on:
      - nameservice
      - dispatcher
```

Deploy with:

```
docker-compose up --scale worker-mytype=5
```

Supported Built-in Types

Current handlers:

Type	Handler	Example
<code>reverse</code>	Reverses a string	<code>"hello"</code> → <code>"olleh"</code>
<code>sum</code>	Sums comma-separated numbers	<code>"1,2,3,4"</code> → <code>"10"</code>
<code>hash</code>	SHA-256 hash of payload	<code>"hello"</code> → <code>"2cf24dba..."</code>
<code>upper</code>	Converts to uppercase	<code>"hello"</code> → <code>"HELLO"</code>
<code>wait</code>	Sleeps N seconds	<code>"3"</code> → sleeps, returns <code>"waited 3s"</code>

See Also

- [Worker Implementation](#) - Base Worker class
- [Handler Registry](#) - All handlers and registration
- [Main Entry Point](#) - TaskWorker that dispatches to handlers

Startanleitung — TaskGrid+

1. Voraussetzungen

Werkzeug	Mindestversion	Zweck
Docker	24.0	Container-Runtime
Docker Compose	2.20 (Plugin)	Orchestrierung aller Dienste
Python	3.12	Nur für lokale Entwicklung ohne Docker
grpcio-tools	aktuell	Nur zum Neuerzeugen der Proto-Bindings

Docker Compose ist ab Docker Desktop 3.x als Plugin enthalten (`docker compose`). Die ältere Standalone-Variante (`docker-compose`) funktioniert ebenfalls.

2. Build-Prozess

2.1 gRPC-Bindings aus `taskgrid.proto` generieren

Die generierten Dateien (`proto/taskgrid_pb2.py` , `proto/taskgrid_pb2_grpc.py`) sind bereits im Repository enthalten und müssen nur neu erzeugt werden, wenn das Schema geändert wird.

```
# Einmalig: grpcio-tools installieren
pip install grpcio-tools

# Proto-Bindings neu generieren (aus dem Repo-Root ausführen)
bash proto/compile_proto.sh
```

Das Skript ruft intern auf:

```
python -m grpc_tools.protoc \
  --proto_path=proto \
  --python_out=proto \
  --grpc_python_out=proto \
  proto/taskgrid.proto
```

2.2 Alle Docker-Images bauen

```
docker compose build
```

Dieser Schritt installiert automatisch alle Abhängigkeiten in die Images. Ein separates `pip install` auf dem Host ist für den produktiven Betrieb nicht nötig.

3. Startanleitung

3.1 Vollständiges System starten

```
docker compose up --build
```

`--build` stellt sicher, dass alle Images vor dem Start aktuell gebaut werden. Beim ersten Start empfehlenswert; danach reicht `docker compose up`.

Startreihenfolge (automatisch via `depends_on` + Healthchecks):

1. Namensdienst (Port 50052)
2. Dispatcher (Port 50051) — wartet bis Namensdienst healthy
3. Worker (`reverse` , `sum` , `hash` , `upper` , `wait`) — warten bis Dispatcher healthy
4. Monitoring — wartet bis Dispatcher und Namensdienst healthy

3.2 Worker skalieren

Mehrere Instanzen desselben Typs können parallel gestartet werden:

```
# 3 Sum-Worker starten
docker compose up --scale worker-sum=3

# Mehrere Typen gleichzeitig skalieren
docker compose up --scale worker-sum=3 --scale worker-reverse=2
```

Der Dispatcher wählt automatisch per Least-Load-Strategie unter den verfügbaren Instanzen aus.

3.3 Aufgabe einreichen (Client)

Der Client ist als einmaliges Kommandozeilenwerkzeug konzipiert (`profiles: [tools]`):

```
# Task senden
docker compose run --rm client send reverse "Hello World"
# Ausgabe: Task submitted: task_id=1

# Ergebnis abfragen
docker compose run --rm client result 1
# Ausgabe: Task 1: COMPLETED — "dlrow olleH"
```

Unterstützte Aufgabentypen:

Typ	Beispiel-Payload	Beispiel-Ergebnis
<code>reverse</code>	"Hello"	"olleH"
<code>sum</code>	"1,2,3,4"	"10"
<code>hash</code>	"hello"	"2cf24dba..." (SHA-256)
<code>upper</code>	"hello"	"HELLO"
<code>wait</code>	"3"	"waited 3s" (3 s Verzögerung)

Typ	Beispiel-Payload	Beispiel-Ergebnis
wordcount	"hello world"	"2"
lower	"HELLO"	"hello"
base64	"hello"	"aGVsbG8="
prime	"17"	"true"

3.4 Monitoring aufrufen

Der Monitoring-Dienst gibt alle `WATCH_INTERVAL_SEC` Sekunden (Standard: 5 s) eine Statustabelle aus:

```
# Logs des Monitoring-Containers verfolgen
docker compose logs -f monitoring
```

Beispielausgabe:

```
+-----+
| TaskGrid+ Status  2026-06-02 12:00:00 |
+-----+
| Workers registered          5|
| Workers active             5|
| Supported task types  hash, reverse, ...|
+-----+
| Tasks queued                0|
| Tasks running               2|
| Tasks completed            42|
| Tasks failed                1|
+-----+
| Avg processing time (ms)    123.4|
| Total timeouts              0|
| Total retries               1|
+-----+
```

Alternativ: Monitoring einmalig manuell abfragen:

```
docker compose run --rm -e WATCH_INTERVAL_SEC=0 monitoring
```

3.5 System stoppen

```
# Alle Container stoppen und entfernen
docker compose down

# Auch Volumes entfernen (löscht Dispatcher-Logs)
docker compose down -v
```

4. Technologiелiste

Kernsprache

Technologie	Version	Zweck
Python	3.12	Implementierungssprache aller Komponenten

Kommunikation

Bibliothek	Version	Zweck
<code>grpcio</code>	aktuell (pip)	gRPC-Laufzeitumgebung — Client/Server-Implementierung
<code>protobuf</code>	aktuell (pip)	Protocol Buffers Serialisierung/Deserialisierung
<code>grpcio-tools</code>	aktuell (pip)	Proto-Compiler (<code>grpc_tools.protoc</code>) — nur zur Entwicklungszeit

Infrastruktur

Technologie	Version	Zweck
Docker	24.0+	Container-Isolation und -Deployment
Docker Compose	2.20+	Orchestrierung, Service-Abhängigkeiten, Skalierung

Standardbibliotheken (keine zusätzliche Installation)

Modul	Zweck
<code>threading</code>	Nebenläufigkeit (Heartbeat-Thread, Task-Threads, Timeout-Checker)
<code>grpc</code>	(Teil von <code>grpcio</code>) gRPC-Kanal und Fehlertypen
<code>hashlib</code>	SHA-256-Berechnung im <code>hash</code> -Worker
<code>base64</code>	Base64-Kodierung im <code>base64</code> -Worker
<code>dataclasses</code>	<code>TaskRecord</code> -Datenstruktur im Dispatcher
<code>collections.deque</code>	Thread-sichere FIFO-Queue
<code>signal</code>	Graceful-Shutdown-Handler (SIGTERM/SIGINT)

5. Lokale Entwicklung (ohne Docker)

Für die Entwicklung einzelner Komponenten außerhalb von Docker:

```
# Abhängigkeiten installieren
pip install grpcio protobuf grpcio-tools

# Namensdienst starten (Terminal 1)
python nameservice/main.py

# Dispatcher starten (Terminal 2)
NAMESERVICE_ADDR=localhost:50052 python dispatcher/main.py

# Worker starten (Terminal 3)
WORKER_TYPE=reverse WORKER_ID=worker-reverse-1 \
NAMESERVICE_ADDRESS=localhost DISPATCHER_ADDRESS=localhost \
python worker/main.py

# Task senden (Terminal 4)
python client/main.py send reverse "Hello World"
python client/main.py result 1
```

6. Konfigurationsreferenz

Alle Komponenten werden ausschließlich über Umgebungsvariablen konfiguriert:

Komponente	Variable	Standard	Beschreibung
Dispatcher	DISPATCHER_PORT	50051	gRPC-Port
Dispatcher	NAMESERVICE_ADDR	nameservice:50052	Adresse des Namensdienstes
Dispatcher	TASK_TIMEOUT_SEC	60	Timeout pro Task in Sekunden
Dispatcher	MAX_RETRIES	3	Maximale Retry-Versuche
Dispatcher	LOG_LEVEL	INFO	Log-Level
Dispatcher	LOG_DIR	(leer)	Optionales Log-Verzeichnis
Namensdienst	NAMESERVICE_PORT	50052	gRPC-Port
Namensdienst	HEARTBEAT_TIMEOUT_SEC	30	Sekunden bis UNHEALTHY
Worker	WORKER_TYPE	(Pflicht)	Aufgabentyp des Workers
Worker	WORKER_PORT	50053	gRPC-Port
Worker	WORKER_ID	(Pflicht)	Eindeutige Worker-ID
Worker	NAMESERVICE_ADDRESS	localhost	Hostname des Namensdienstes
Worker	DISPATCHER_ADDRESS	localhost	Hostname des Dispatchers
Worker	HEARTBEAT_INTERVAL_SEC	10	Heartbeat-Intervall in Sekunden
Monitoring	DISPATCHER_ADDR	localhost:50051	Adresse des Dispatchers

Komponente	Variable	Standard	Beschreibung
Monitoring	NAMESERVICE_ADDR	localhost:50052	Adresse des Namensdienstes
Monitoring	WATCH_INTERVAL_SEC	0	Abfrageintervall (0 = einmalig)
Client	DISPATCHER_HOST	localhost	Hostname des Dispatchers
Client	DISPATCHER_PORT	50051	Port des Dispatchers

Docker Build Guide

Overview

Each TaskGrid+ component has its own Dockerfile. All images use `python:3.12-slim` and are configured exclusively via environment variables — no addresses or worker types are hardcoded.

The pre-generated proto bindings (`proto/taskgrid_pb2.py` , `proto/taskgrid_pb2_grpc.py`) are copied into each image at build time. No `protoc` installation is required inside the containers.

Building all images

```
docker compose build
```

Running the full stack

```
docker compose up
```

This starts: nameservice, dispatcher, all five worker types (sum, reverse, hash, upper, wait), and the monitoring dashboard.

Environment variables

nameservice

Variable	Default	Description
NAMESERVICE_PORT	50051	gRPC listen port
HEARTBEAT_TIMEOUT_SEC	30	Seconds before a worker is marked UNHEALTHY
OFFLINE_TIMEOUT_SEC	90	Seconds before a worker is marked OFFLINE

dispatcher

Variable	Default	Description
DISPATCHER_PORT	50051	gRPC listen port
NAMESERVICE_ADDR	nameservice:50052	Namensdienst address
TASK_TIMEOUT_SEC	60	Task timeout before retry
MAX_RETRIES	3	Max dispatch retries per task
LOG_LEVEL	INFO	Log verbosity (DEBUG/INFO/WARNING/ERROR)

worker

Variable	Default	Description
WORKER_ID	(required)	Unique worker identifier
WORKER_TYPE	(required)	Task type: <code>sum</code> , <code>reverse</code> , <code>hash</code> , <code>upper</code> , <code>wait</code>
WORKER_ADDRESS	0.0.0.0	Address registered with the Namensdienst (set to the service hostname in Docker)
WORKER_PORT	50052	gRPC listen port
NAMESERVICE_ADDRESS	localhost	Namensdienst host
NAMESERVICE_PORT	50051	Namensdienst port
DISPATCHER_ADDRESS	localhost	Dispatcher host (for result return)
DISPATCHER_PORT	50051	Dispatcher port
WORKER_CAPACITY	10	Max concurrent tasks

monitoring

Variable	Default	Description
DISPATCHER_ADDR	localhost:50051	Dispatcher address
NAMESERVICE_ADDR	localhost:50052	Namensdienst address
WATCH_INTERVAL_SEC	0	Refresh interval in seconds; 0 = run once

client

Variable	Default	Description
DISPATCHER_HOST	localhost	Dispatcher host
DISPATCHER_PORT	50051	Dispatcher port

Sending a task via client container

```
docker compose run --rm client send sum "1,2,3,4"
docker compose run --rm client result 1
```

The `client` service is in the `tools` profile and does not start with `docker compose up`. Use `docker compose run` to invoke it on demand.

Running the monitoring dashboard standalone

```
docker compose run --rm monitoring
```

Adding more workers of a given type

To run multiple workers of the same type, add additional service entries in `docker-compose.yml` with unique `WORKER_ID` and `WORKER_ADDRESS` values:

```
worker-sum-2:
  build:
    context: .
    dockerfile: worker/Dockerfile
  environment:
    - WORKER_ID=worker-sum-2
    - WORKER_TYPE=sum
    - WORKER_ADDRESS=worker-sum-2
    - WORKER_PORT=50053
    - NAMESERVICE_ADDRESS=nameservice
    - NAMESERVICE_PORT=50052
    - DISPATCHER_ADDRESS=dispatcher
    - DISPATCHER_PORT=50051
  depends_on:
    - nameservice
    - dispatcher
```

Each worker must have a unique `WORKER_ID` and its `WORKER_ADDRESS` must match the Docker Compose service name so the dispatcher can route tasks to it.

Testprotokoll — Erfolgreiche Aufgaben

Datum: 2026-06-04

System: TaskGrid+

Umgebung: Docker Compose (Podman rootless), lokale Maschine

Systemzustand beim Test

Komponente	Container-ID	IP-Adresse
Nameservice	taskgrid-plus-nameservice-1	10.89.0.10
Dispatcher	taskgrid-plus-dispatcher-1	10.89.0.11
Worker <code>reverse</code>	<code>58c35f0a9f88</code>	10.89.0.13
Worker <code>sum</code>	<code>f394568ef865</code>	10.89.0.14
Worker <code>hash</code>	<code>0e0c726b9706</code>	10.89.0.15
Worker <code>upper</code>	<code>a276953086f5</code>	10.89.0.16
Worker <code>wait</code>	<code>08fd4519f2fa</code>	10.89.0.17

Alle 5 Worker wurden beim Nameservice registriert.

Dispatcher-Status vor dem Test: `workers_registered=5`, `workers_active=5`, `tasks_completed=0`

Testablauf und Ergebnisse

```
TaskGrid+ - Testprotokoll Erfolgreiche Tests

$ GRPC_DNS_RESOLVER=native python3 run_test_protocol.py

=====
TASKGRID+ TEST PROTOCOL - 2026-06-04
=====

[12:04:15.340] TC-01 POST type='reverse' payload='hello'
- success=True task_id=1 msg='Task queued successfully'
[12:04:15.643] TC-02 POST type='sum' payload='1,2,3,4'
- success=True task_id=2 msg='Task queued successfully'
[12:04:15.948] TC-03 POST type='hash' payload='hello'
- success=True task_id=3 msg='Task queued successfully'
[12:04:16.252] TC-04 POST type='upper' payload='hello world'
- success=True task_id=4 msg='Task queued successfully'
[12:04:16.559] TC-05 POST type='wait' payload='2'
- success=True task_id=5 msg='Task queued successfully'
[12:04:16.864] TC-06 POST type='reverse' payload='taskgrid'
- success=True task_id=6 msg='Task queued successfully'

Polling for results ...

[12:04:17.168] TC-01 task_id=1 COMPLETED result='olleh'
[12:04:17.170] TC-02 task_id=2 COMPLETED result='10'
[12:04:17.172] TC-03 task_id=3 COMPLETED
result='2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824'
[12:04:17.173] TC-04 task_id=4 COMPLETED result='HELLO WORLD'
[12:04:17.176] TC-06 task_id=6 COMPLETED result='dirgksat'
[12:04:19.184] TC-05 task_id=5 COMPLETED result='waited 2s'

=====
SUMMARY: 6/6 tasks completed
=====

[] TC-01 id=1 reverse payload='hello' result='olleh' (1828 ms)
[] TC-02 id=2 sum payload='1,2,3,4' result='10' (1527 ms)
[] TC-03 id=3 hash payload='hello' result='2cf24dba...' (1224 ms)
[] TC-04 id=4 upper payload='hello world' result='HELLO WORLD' (921 ms)
[] TC-05 id=5 wait payload='2' result='waited 2s' (2625 ms)
[] TC-06 id=6 reverse payload='taskgrid' result='dirgksat' (312 ms)
```

Testfälle

TC-01 — reverse „hello“

Feld	Wert
Typ	reverse
Payload	hello
task_id	1
Erwartetes Ergebnis	olleh
Tatsächliches Ergebnis	olleh ✓
Bearbeitender Worker	58c35f0a9f88 (10.89.0.13)
Laufzeit (Worker)	5 ms
End-to-End-Zeit	1828 ms

Status-Verlauf:

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567455331 task_id=1 status=CREATED type=reverse
[dispatcher] request_id=proto-1780567455331 task_id=1 status=QUEUED type=reverse
[dispatcher] request_id=8f887a76-4d6a-47dc-b54b-de8e6fe9f37e task_id=1 status=DISPATCHED worker=58c35f0a9f88
[dispatcher] request_id=8f887a76-4d6a-47dc-b54b-de8e6fe9f37e task_id=1 status=PROCESSING worker=58c35f0a9f88
[dispatcher] request_id=19a03998-14f3-4c65-8e3f-ab3b0e61f2da task_id=1 status=COMPLETED duration_ms=5
[dispatcher] request_id=19a03998-14f3-4c65-8e3f-ab3b0e61f2da task_id=1 event=RESULT_RECEIVED worker=58c35f0a9f88
```

Worker-Logs (worker=reverse):

```
[58c35f0a9f88] request_id=8f887a76-4d6a-47dc-b54b-de8e6fe9f37e task_id=1 status=RECEIVED type=reverse
[58c35f0a9f88] request_id=8f887a76-4d6a-47dc-b54b-de8e6fe9f37e task_id=1 status=PROCESSING type=reverse
[58c35f0a9f88] request_id=8f887a76-4d6a-47dc-b54b-de8e6fe9f37e task_id=1 status=COMPLETED
```

TC-02 — sum „1,2,3,4“

Feld	Wert
Typ	sum
Payload	1,2,3,4
task_id	2
Erwartetes Ergebnis	10
Tatsächliches Ergebnis	10 ✓
Bearbeitender Worker	f394568ef865 (10.89.0.14)
Laufzeit (Worker)	6 ms
End-to-End-Zeit	1527 ms

Status-Verlauf:

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567455640 task_id=2 status=CREATED type=sum
[dispatcher] request_id=proto-1780567455640 task_id=2 status=QUEUED type=sum
[dispatcher] request_id=2928e445-7051-48f4-b178-f9dcf87e0670 task_id=2 status=DISPATCHED worker=f394568ef865
[dispatcher] request_id=2928e445-7051-48f4-b178-f9dcf87e0670 task_id=2 status=PROCESSING worker=f394568ef865
[dispatcher] request_id=071f9a2f-3879-406f-8fea-00b4828772de task_id=2 status=COMPLETED duration_ms=6
[dispatcher] request_id=071f9a2f-3879-406f-8fea-00b4828772de task_id=2 event=RESULT_RECEIVED worker=f394568ef865
```

Worker-Logs (worker-sum):

```
[f394568ef865] request_id=2928e445-7051-48f4-b178-f9dcf87e0670 task_id=2 status=RECEIVED type=sum
[f394568ef865] request_id=2928e445-7051-48f4-b178-f9dcf87e0670 task_id=2 status=PROCESSING type=sum
[f394568ef865] request_id=2928e445-7051-48f4-b178-f9dcf87e0670 task_id=2 status=COMPLETED
```

TC-03 — hash „hello“

Feld	Wert
Typ	hash
Payload	hello
task_id	3
Erwartetes Ergebnis	SHA-256 von „hello“
Tatsächliches Ergebnis	2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 ✓
Bearbeitender Worker	0e0c726b9706 (10.89.0.15)
Laufzeit (Worker)	5 ms
End-to-End-Zeit	1224 ms

Status-Verlauf:

```
CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED
```

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567455943 task_id=3 status=CREATED type=hash
[dispatcher] request_id=proto-1780567455943 task_id=3 status=QUEUED type=hash
[dispatcher] request_id=04b45709-0f7d-4ea2-81f7-456755b4e31a task_id=3 status=DISPATCHED worker=0e0c726b9706
[dispatcher] request_id=04b45709-0f7d-4ea2-81f7-456755b4e31a task_id=3 status=PROCESSING worker=0e0c726b9706
[dispatcher] request_id=09002f39-0e99-4b3f-a2a3-05ab89647537 task_id=3 status=COMPLETED duration_ms=5
[dispatcher] request_id=09002f39-0e99-4b3f-a2a3-05ab89647537 task_id=3 event=RESULT_RECEIVED worker=0e0c726b9706
```

Worker-Logs (worker-hash):

```
[0e0c726b9706] request_id=04b45709-0f7d-4ea2-81f7-456755b4e31a task_id=3 status=RECEIVED type=hash
[0e0c726b9706] request_id=04b45709-0f7d-4ea2-81f7-456755b4e31a task_id=3 status=PROCESSING type=hash
[0e0c726b9706] request_id=04b45709-0f7d-4ea2-81f7-456755b4e31a task_id=3 status=COMPLETED
```

TC-04 — `upper` „hello world“

Feld	Wert
Typ	<code>upper</code>
Payload	<code>hello world</code>
task_id	4
Erwartetes Ergebnis	HELLO WORLD
Tatsächliches Ergebnis	HELLO WORLD ✓
Bearbeitender Worker	a276953086f5 (10.89.0.16)
Laufzeit (Worker)	9 ms
End-to-End-Zeit	921 ms

Status-Verlauf:

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567456249 task_id=4 status=CREATED type=upper
[dispatcher] request_id=proto-1780567456249 task_id=4 status=QUEUED type=upper
[dispatcher] request_id=c1d7a2fd-96e9-46c6-9fbb-92127b0c1315 task_id=4 status=DISPATCHED worker=a276953086f5
[dispatcher] request_id=c1d7a2fd-96e9-46c6-9fbb-92127b0c1315 task_id=4 status=PROCESSING worker=a276953086f5
[dispatcher] request_id=93d89078-0b1d-4bbe-81a6-060e1f6f53b8 task_id=4 status=COMPLETED duration_ms=9
[dispatcher] request_id=93d89078-0b1d-4bbe-81a6-060e1f6f53b8 task_id=4 event=RESULT_RECEIVED worker=a276953086f5
```

Worker-Logs (`worker-upper`):

```
[a276953086f5] request_id=c1d7a2fd-96e9-46c6-9fbb-92127b0c1315 task_id=4 status=RECEIVED type=upper
[a276953086f5] request_id=c1d7a2fd-96e9-46c6-9fbb-92127b0c1315 task_id=4 status=PROCESSING type=upper
[a276953086f5] request_id=c1d7a2fd-96e9-46c6-9fbb-92127b0c1315 task_id=4 status=COMPLETED
```

TC-05 — `wait` „2“

Feld	Wert
Typ	<code>wait</code>
Payload	2
task_id	5
Erwartetes Ergebnis	<code>waited 2s</code>
Tatsächliches Ergebnis	<code>waited 2s</code> ✓

Feld	Wert
Bearbeitender Worker	08fd4519f2fa (10.89.0.17)
Laufzeit (Worker)	2013 ms
End-to-End-Zeit	2625 ms

Status-Verlauf:

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567456552 task_id=5 status=CREATED type=wait
[dispatcher] request_id=proto-1780567456552 task_id=5 status=QUEUED type=wait
[dispatcher] request_id=8432ec8a-0935-4f6d-bb18-0224ee116948 task_id=5 status=DISPATCHED worker=08fd4519f2fa
[dispatcher] request_id=8432ec8a-0935-4f6d-bb18-0224ee116948 task_id=5 status=PROCESSING worker=08fd4519f2fa
[dispatcher] request_id=cec41f48-ee7f-4b94-a235-bb0960e060cc task_id=5 status=COMPLETED duration_ms=2013
[dispatcher] request_id=cec41f48-ee7f-4b94-a235-bb0960e060cc task_id=5 event=RESULT_RECEIVED worker=08fd4519f2
```

Worker-Logs (worker-wait):

```
[08fd4519f2fa] request_id=8432ec8a-0935-4f6d-bb18-0224ee116948 task_id=5 status=RECEIVED type=wait
[08fd4519f2fa] request_id=8432ec8a-0935-4f6d-bb18-0224ee116948 task_id=5 status=PROCESSING type=wait
[08fd4519f2fa] request_id=8432ec8a-0935-4f6d-bb18-0224ee116948 task_id=5 status=COMPLETED
```

TC-06 — `reverse` „taskgrid“

Feld	Wert
Typ	<code>reverse</code>
Payload	<code>taskgrid</code>
task_id	<code>6</code>
Erwartetes Ergebnis	<code>dirgksat</code>
Tatsächliches Ergebnis	<code>dirgksat</code> ✓
Bearbeitender Worker	58c35f0a9f88 (10.89.0.13)
Laufzeit (Worker)	2 ms
End-to-End-Zeit	312 ms

Status-Verlauf:

CREATED → QUEUED → DISPATCHED → PROCESSING → COMPLETED

Dispatcher-Logs:

```
[dispatcher] request_id=proto-1780567456859 task_id=6 status=CREATED type=reverse
[dispatcher] request_id=proto-1780567456859 task_id=6 status=QUEUED type=reverse
[dispatcher] request_id=69b95574-355c-4ec6-b767-3f7f6d490335 task_id=6 status=DISPATCHED worker=58c35f0a9f88
[dispatcher] request_id=69b95574-355c-4ec6-b767-3f7f6d490335 task_id=6 status=PROCESSING worker=58c35f0a9f88
[dispatcher] request_id=c7f5e104-a9a3-4e66-812a-099a4b9f5cd5 task_id=6 status=COMPLETED duration_ms=2
[dispatcher] request_id=c7f5e104-a9a3-4e66-812a-099a4b9f5cd5 task_id=6 event=RESULT_RECEIVED worker=58c35f0a9f88
```

Worker-Logs (worker=reverse):

```
[58c35f0a9f88] request_id=69b95574-355c-4ec6-b767-3f7f6d490335 task_id=6 status=RECEIVED type=reverse
[58c35f0a9f88] request_id=69b95574-355c-4ec6-b767-3f7f6d490335 task_id=6 status=PROCESSING type=reverse
[58c35f0a9f88] request_id=69b95574-355c-4ec6-b767-3f7f6d490335 task_id=6 status=COMPLETED
```

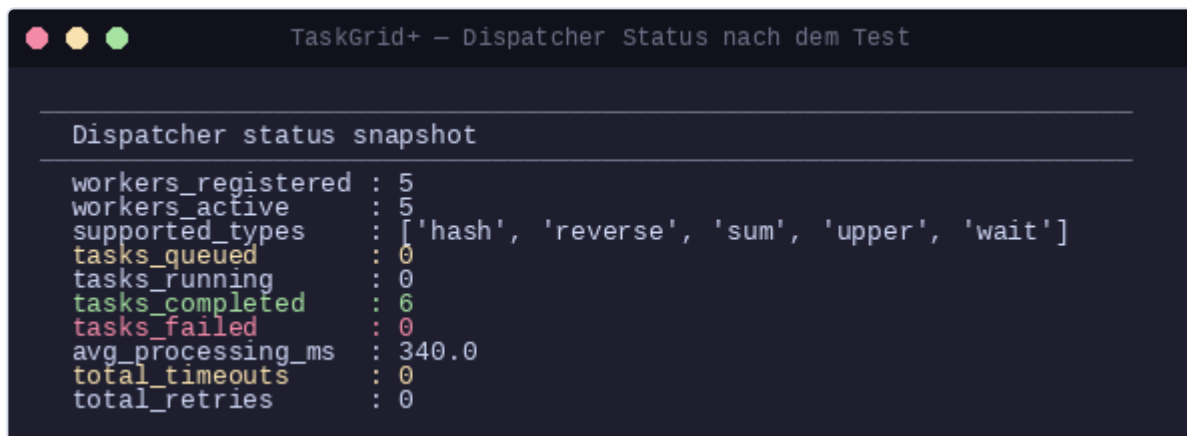
Gesamtübersicht

TC	Typ	Payload	Erwartet	Erhalten	Worker	Status
TC-01	reverse	hello	olleh	olleh	58c35f0a9f88	COMPLETED ✓
TC-02	sum	1,2,3,4	10	10	f394568ef865	COMPLETED ✓
TC-03	hash	hello	SHA-256-Hex	2cf24dba...b9824	0e0c726b9706	COMPLETED ✓
TC-04	upper	hello world	HELLO WORLD	HELLO WORLD	a276953086f5	COMPLETED ✓
TC-05	wait	2	waited 2s	waited 2s	08fd4519f2fa	COMPLETED ✓
TC-06	reverse	taskgrid	dirgksat	dirgksat	58c35f0a9f88	COMPLETED ✓

Ergebnis: 6/6 Aufgaben erfolgreich abgeschlossen.

Dispatcher-Status nach dem Test

```
workers_registered : 5
workers_active     : 5
supported_types    : [hash, reverse, sum, upper, wait]
tasks_queued       : 0
tasks_running      : 0
tasks_completed    : 6
tasks_failed       : 0
avg_processing_ms   : 340.0
total_timeouts     : 0
total_retries      : 0
```

A terminal window titled "TaskGrid+ - Dispatcher Status nach dem Test" displays a "Dispatcher status snapshot". The snapshot lists various metrics: 5 workers registered and active, supported types including hash, reverse, sum, upper, and wait, 0 tasks queued and running, 6 tasks completed, 0 tasks failed, an average processing time of 340.0 ms, 0 total timeouts, and 0 total retries. The metrics are color-coded: 'tasks_completed' is green, 'tasks_failed' is red, and 'total_timeouts' is yellow.

```
TaskGrid+ - Dispatcher Status nach dem Test

Dispatcher status snapshot
workers_registered : 5
workers_active    : 5
supported_types   : ['hash', 'reverse', 'sum', 'upper', 'wait']
tasks_queued      : 0
tasks_running     : 0
tasks_completed   : 6
tasks_failed      : 0
avg_processing_ms : 340.0
total_timeouts    : 0
total_retries     : 0
```

Testprotokoll — Fehler- und Robustheitstests

Datum: 2026-06-04

System: TaskGrid+

Umgebung: Docker Compose (Podman rootless), lokale Maschine

Mindestanforderung gemäß PDF §16.2: ≥ 3 Fehlerfälle.

Dokumentiert werden die Fälle **A**, **C** und **B**.

Fall A — Unbekannter Task-Typ

Ausgangssituation

Alle 5 Worker (reverse, sum, hash, upper, wait) sind beim Nameservice registriert und aktiv.

Durchgeführte Aktion

Ein Task mit `type="unknowntype"` und `payload="test"` wird über `PostTask` an den Dispatcher gesendet.

Erwartetes Verhalten

Der Dispatcher lehnt den Task sofort ab (kein Worker für diesen Typ registriert), gibt `success=false` zurück, vergibt keine `task_id`.

Tatsächliches Verhalten

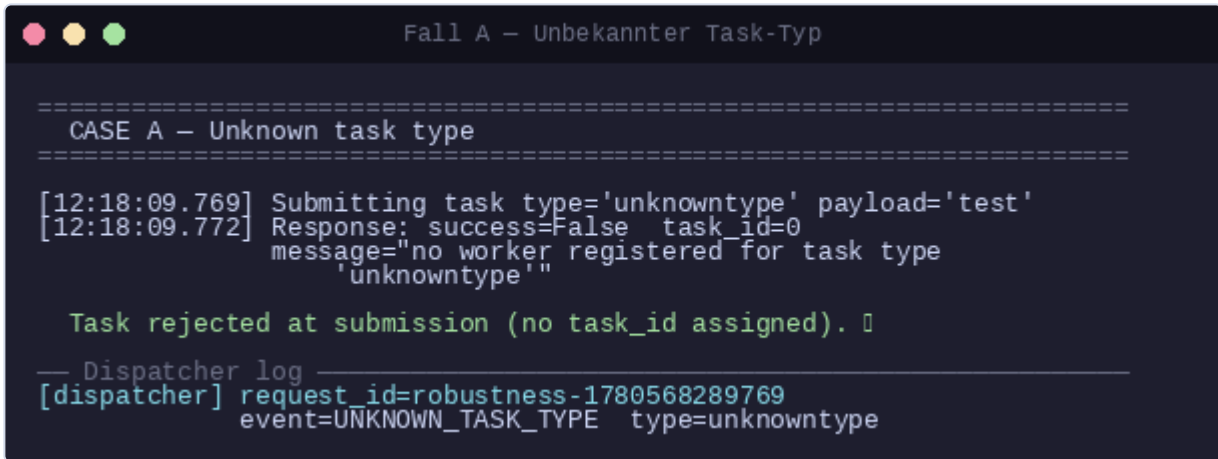
Der Dispatcher lehnt den Task ohne Zuweisung einer `task_id` ab:

```
success=False task_id=0 message="no worker registered for task type 'unknowntype'"
```

Kein Eintrag im internen Task-Store, kein Status-Übergang. Das System läuft ohne Unterbrechung weiter.

Dispatcher-Log

```
[dispatcher] request_id=robustness-1780568289769 event=UNKNOWN_TASK_TYPE type=unknowntype
```



```

=====
CASE A – Unknown task type
=====
[12:18:09.769] Submitting task type='unknowntype' payload='test'
[12:18:09.772] Response: success=False task_id=0
                message="no worker registered for task type
                'unknowntype'"

Task rejected at submission (no task_id assigned). 🚫

— Dispatcher log —
[dispatcher] request_id=robustness-1780568289769
                event=UNKNOWN_TASK_TYPE type=unknowntype

```

Bewertung

Korrekt. Der Dispatcher erkennt sofort, dass kein Worker für den Typ existiert, und gibt einen klaren Fehler zurück. Dem Client wird eine verständliche Fehlermeldung geliefert. Keine `task_id` wurde vergeben, es findet kein Status-Übergang statt.

Fall C — Kein Worker für den gefragten Typ verfügbar

Ausgangssituation

Worker `worker-sum` (`f394568ef865` , IP 10.89.0.12) ist der einzige registrierte Worker für den Typ `sum` .

Durchgeführte Aktion

1. Container `worker-sum` wird gestoppt (`podman stop taskgrid-plus-worker-sum-1`)
2. Direkt danach wird ein Task `type="sum"` , `payload="1,2,3"` eingereicht
3. Der Task wird 15 Sekunden lang beobachtet
4. Anschließend wird `worker-sum` neu gestartet

Erwartetes Verhalten

Der Dispatcher nimmt den Task in die Warteschlange auf (weil er zum Zeitpunkt der Einreichung noch im Nameservice-Cache sehen kann, dass ein `sum`-Worker registriert ist). Dispatch-Versuche schlagen fehl. Nach dem Neustart des Workers wird der Task erfolgreich bearbeitet.

Tatsächliches Verhalten

Einreichung (`task_id=7`):

```
[12:18:12.945] success=True task_id=7 message='Task queued successfully'
```

Status während Worker offline (15 s):

```
[12:18:12.947] task_id=7 QUEUED
[12:18:15.950] task_id=7 DISPATCHED ← Dispatch-Versuch an gestoppten Worker
[12:18:18.952] task_id=7 DISPATCHED ← Wiederholter Versuch
[12:18:21.954] task_id=7 DISPATCHED
[12:18:24.956] task_id=7 QUEUED ← Zurück in die Queue nach Fehlversuchen
```

Nach Neustart des Workers:

```
[12:18:33.073] task_id=7 COMPLETED result='6'
```

Dispatcher-Logs (Auszug)

```
[dispatcher] request_id=... task_id=7 status=CREATED type=sum
[dispatcher] request_id=... task_id=7 status=QUEUED type=sum
[dispatcher] request_id=ef91e19d-... task_id=7 status=DISPATCHED worker=f394568ef865
[dispatcher] request_id=ef91e19d-... task_id=7 event=WORKER_UNREACHABLE worker=f394568ef865
error=failed to connect to all addresses; last error: UNKNOWN: ipv4:10.89.0.12:50053:
Failed to connect to remote host: getsockopt(SO_ERROR): No route to host
[dispatcher] request_id=... task_id=7 status=DISPATCHED worker=f394568ef865
[dispatcher] request_id=... task_id=7 event=WORKER_UNREACHABLE worker=f394568ef865 [...]
... (weitere Versuche) ...
[dispatcher] request_id=fd84f32b-... task_id=7 status=COMPLETED duration_ms=2
```

```
Fall C - Kein Worker verfügbar

=====
CASE C - No worker available for task type
=====

[12:18:09.772] Stopping worker-sum container...
[12:18:12.942] worker-sum stopped.

[12:18:12.942] Submitting task type='sum' payload='1,2,3'
[12:18:12.945] Response: success=True task_id=7 msg='Task queued successfully'

Task accepted (id=7): Polling while worker offline...
[12:18:12.947] task_id=7 QUEUED
[12:18:15.950] task_id=7 DISPATCHED ← attempt to stopped worker
[12:18:18.952] task_id=7 DISPATCHED ← retry
[12:18:21.954] task_id=7 DISPATCHED ← retry
[12:18:24.956] task_id=7 QUEUED ← back in queue after failures

[12:18:27.956] Restarting worker-sum...
[12:18:33.070] worker-sum restarted.
[12:18:33.073] task_id=7 COMPLETED result='6' □
```

Bewertung

Weitgehend korrekt. Der Dispatcher hält den Task in der Warteschlange und wiederholt den Dispatch-Versuch, statt sofort zu scheitern. Nach dem Neustart des Workers wird der Task automatisch zugestellt und erfolgreich abgeschlossen (`result='6'` = 1+2+3). Das System zeigt damit Selbstheilungsfähigkeit. Verbesserungspotenzial: Der Dispatcher sollte den Nameservice öfter befragen, um offline gegangene Worker früher aus dem Dispatch-Pool zu entfernen.

Fall B — Worker-Absturz während der Verarbeitung

Ausgangssituation

Worker `worker-wait` (`08fd4519f2fa`) ist der einzige registrierte Worker für den Typ `wait` .
`TASK_TIMEOUT_SEC=60` .

Durchgeführte Aktion

1. Ein Task `type="wait"` , `payload="10"` (10-Sekunden-Pause) wird eingereicht
2. Sobald der Task den Status `PROCESSING` erreicht, wird der Container `worker-wait` gestoppt
3. Das System wird bis zur Endentscheidung des Dispatchers beobachtet

Erwartetes Verhalten

Da der Worker abstürzt, ohne ein Ergebnis zurückzugeben, wartet der Dispatcher bis `TASK_TIMEOUT_SEC` (60 s). Anschließend wird ein Retry-Versuch unternommen. Da kein alternativer `wait` -Worker verfügbar ist, wird der Task mit `FAILED` markiert. Das restliche System bleibt betriebsbereit.

Tatsächliches Verhalten

Einreichung und sofortiger Dispatch (`task_id=8`):

```
[12:18:33.076] success=True task_id=8 message='Task queued successfully'
[12:18:33.077] task_id=8 QUEUED
[12:18:34.081] task_id=8 PROCESSING ← Worker hat Task übernommen
```

Worker-Kill:

```
[12:18:34.082] worker-wait gestoppt (Container podman stop)
```

Timeout und Fehlschlag (nach ~64 s):

```
[12:18:34.234] - [12:19:38.331] task_id=8 PROCESSING (Worker antwortet nicht)
[12:19:38.331] task_id=8 FAILED message='no alternative worker available for retry'
```

Dispatcher-Logs

```
[dispatcher] request_id=... task_id=8 status=CREATED type=wait
[dispatcher] request_id=... task_id=8 status=QUEUED type=wait
[dispatcher] request_id=20927f29-... task_id=8 status=DISPATCHED worker=08fd4519f2fa
[dispatcher] request_id=20927f29-... task_id=8 status=PROCESSING worker=08fd4519f2fa
[dispatcher] request_id=c47ee937-... task_id=8 status=TIMEOUT worker=08fd4519f2fa elapsed_ms=64210
[dispatcher] request_id=c47ee937-... task_id=8 status=RETRYING retry_count=1
[dispatcher] request_id=841f037e-... task_id=8 event=NO_WORKER type=wait
[dispatcher] request_id=841f037e-... task_id=8 status=FAILED error=no alternative worker available for retr
```

Status-Verlauf:

```
CREATED → QUEUED → DISPATCHED → PROCESSING → TIMEOUT → RETRYING → FAILED
```

```

Fall B – Worker-Absturz während der Verarbeitung

=====
CASE B – Worker crash during processing
=====

[12:18:33.073] Submitting wait task (10 s)...
[12:18:33.076] Response: success=True task_id=8 msg='Task queued successfully'

Waiting for PROCESSING state...
[12:18:33.077] task_id=8 QUEUED
[12:18:34.081] task_id=8 PROCESSING – worker has the task

[12:18:34.082] Killing worker-wait container mid-task...
[12:18:34.233] worker-wait stopped.

Polling for timeout / failure...
[12:18:34] – [12:19:32] task_id=8 PROCESSING (worker silent)
[12:19:38.331] task_id=8 FAILED
message='no alternative worker available for retry'

— Dispatcher log —
[dispatcher] task_id=8 status=TIMEOUT worker=08fd4519f2fa elapsed_ms=64210
[dispatcher] task_id=8 status=RETRYING retry_count=1
[dispatcher] task_id=8 event=NO_WORKER type=wait
[dispatcher] task_id=8 status=FAILED error=no alternative worker available

Status: CREATED-QUEUED-DISPATCHED-PROCESSING-TIMEOUT-RETRYING-FAILED

```

Bewertung

Korrekt. Der Dispatcher erkennt nach 64 s ($\approx \text{TASK_TIMEOUT_SEC}=60 + \text{Puffer}$), dass der Worker nicht antwortet, und setzt den Status auf `TIMEOUT`. Es wird ein Retry versucht, der wegen fehlender alternativer Worker direkt zu `FAILED` führt. Die Fehlermeldung ist eindeutig. Alle anderen Worker und der Dispatcher bleiben betriebsbereit. Das Verhalten entspricht der spezifizierten Fehlerbehandlung.

Gesamtbewertung

Fall	Beschreibung	Erwartetes Verhalten eingetreten?
A	Unbekannter Task-Typ	Ja — sofortige Ablehnung, keine task_id ✓
C	Worker offline bei Einreichung	Ja — Task bleibt in Queue, nach Neustart erledigt ✓
B	Worker-Absturz während Verarbeitung	Ja — TIMEOUT → RETRY → FAILED, System läuft weiter ✓

Das System verhält sich in allen drei Fehlerfällen erwartungsgemäß und robust. Kein Fehler führt zum Absturz des Dispatchers oder anderer Komponenten.

ADR-001: Kommunikationsprotokoll — gRPC statt UDP

Kontext

Alle Komponenten (Client, Dispatcher, Namensdienst, Worker, Monitoring) müssen miteinander kommunizieren. Es war zu entscheiden, welches Transportprotokoll und welches Serialisierungsformat eingesetzt werden. Die Anforderungen umfassen typsichere Schnittstellen, Zuverlässigkeit, Skalierbarkeit und Unterstützung mehrerer Sprachen.

Entscheidung

gRPC (HTTP/2 + Protocol Buffers) wird als einziges Kommunikationsprotokoll im System eingesetzt. Alle Dienste exponieren gRPC-Endpoints; alle Nachrichten sind als Protobuf-Messages in `taskgrid.proto` definiert.

Alternativen

Alternative	Bewertung
Rohes UDP	Kein Verbindungsaufbau, minimale Latenz — aber kein Zustellgarantie, kein Framing, kein automatisches Retry. Eigene Implementierung für Paketverlust, Reihenfolge und Serialisierung nötig.
REST/ HTTP+JSON	Weit verbreitet, leicht debuggbar — aber keine strenge Typisierung, höherer Overhead durch Text-Serialisierung, kein natives Streaming.
ZeroMQ / Nanomsg	Hohe Performance, flexible Topologien — aber kein Code-Generator, keine standardisierte Schnittstellenbeschreibung, weniger Ökosystem-Support.
gRPC (gewählt)	Typsichere, aus <code>.proto</code> generierte Stubs, zuverlässige Übertragung (TCP), bidirektionales Streaming möglich, sprachunabhängig.

Konsequenzen

Vorteile:

- Die `.proto`-Datei ist gleichzeitig die vollständige, versionierbare Schnittstellendokumentation.
- Generierter Code (Stubs) verhindert Serialisierungsfehler zur Laufzeit.
- HTTP/2-Multiplexing ermöglicht parallele Anfragen über eine einzige Verbindung.
- Verbindungsfehler liefern typisierte `grpc.RpcError`-Ausnahmen — einheitliches Fehlerhandling.

Nachteile:

- Protobuf-Kompilierungsschritt erforderlich (`compile_proto.sh`).
- Binary-Format erschert direktes Debugging ohne Tools (z. B. `grpcurl`).
- Höherer initialer Setup-Aufwand gegenüber simplem JSON über HTTP.

ADR-002: Nachrichtenformat — Protocol Buffers statt JSON

Kontext

Für die Serialisierung der gRPC-Nachrichten war zu entscheiden, ob Protocol Buffers (Protobuf) oder ein alternatives Format (z. B. JSON, MessagePack) eingesetzt wird. Das Format beeinflusst Typsicherheit, Performance, Versionierbarkeit und Lesbarkeit.

Entscheidung

Protocol Buffers v3 wird für alle Nachrichten verwendet. Das Schema ist in `proto/taskgrid.proto` definiert und versioniert. Stubs für Python werden per `compile_proto.sh` generiert.

Alternativen

Alternative	Bewertung
JSON über HTTP	Menschenlesbar, einfach zu debuggen — aber kein Schema, keine Typprüfung zur Compile-Zeit, höherer Speicher- und CPU-Overhead durch Text-Parsing.
JSON über gRPC (gRPC-Gateway)	Ermöglicht REST-Clients — aber doppelter Serialisierungsschritt, höhere Latenz, kein Vorteil gegenüber nativem Protobuf bei rein internen Diensten.
MessagePack	Kompaktes Binärformat — aber kein Schema, kein Code-Generator, keine native gRPC-Integration.
Protocol Buffers (gewählt)	Stark typisiert, kompakt, versionierbar, native gRPC-Integration, Code-Generierung für alle gängigen Sprachen.

Konsequenzen

Vorteile:

- Das `taskgrid.proto`-Schema dient als Single Source of Truth für alle Schnittstellen.
- Fehlende Pflichtfelder oder falsche Typen werden bereits beim Serialisieren erkannt.
- Binärformat ist ca. 3–10× kompakter als äquivalentes JSON — relevant bei vielen parallelen Workern.
- Neue Felder können mit neuen Feldnummern ergänzt werden, ohne bestehende Clients zu brechen (Vorwärtskompatibilität).

Nachteile:

- Nachrichten sind ohne Deserialisierungs-Tool nicht direkt lesbar.
- Schema-Änderungen erfordern erneute Code-Generierung und Verteilung der Stubs an alle Komponenten.
- Feldnummern dürfen nicht geändert oder wiederverwendet werden — erfordert Disziplin bei Schemaevolution.

ADR-003: Worker-Auswahlstrategie — Least Load

Kontext

Der Dispatcher empfängt Tasks und muss unter ggf. mehreren verfügbaren Workern desselben Typs genau einen auswählen. Die Wahl der Strategie beeinflusst Lastverteilung, Fehlertoleranz und Implementierungskomplexität. Worker melden ihren aktuellen `current_load` (Anzahl laufender Tasks) bei jedem Heartbeat an den Namensdienst.

Entscheidung

Least Load — der Worker mit dem niedrigsten gemeldeten `current_load` -Wert wird bevorzugt.

Ablauf in `_select_worker` :

1. `LookupWorker` am Namensdienst liefert alle registrierten Worker des gewünschten Typs.
2. Nur Worker mit Status `ACTIVE` werden berücksichtigt.
3. Bei einem Retry-Versuch wird der zuletzt fehlgeschlagene Worker ausgeschlossen (`last_failed_worker`), sofern ein anderer verfügbar ist.
4. `min(candidates, key=lambda w: w.current_load)` wählt den am wenigsten ausgelasteten Worker.

Alternativen

Strategie	Bewertung
Round-robin	Einfach, kein Zustand — ignoriert aber unterschiedliche Worker-Kapazitäten und aktuelle Last.
Random	Keine Koordination — führt zu ungleichmäßiger Auslastung bei heterogenen Workern.
Least Load (gewählt)	Nutzt den bereits im <code>Worker</code> -Proto vorhandenen <code>current_load</code> -Wert; kein zusätzlicher Dispatcher-Zustand.
Weighted Round-robin	Berücksichtigt Kapazitäten — erfordert aber persistenten Zähler pro Worker im Dispatcher.

Konsequenzen

Vorteile:

- Tasks fließen automatisch zu weniger ausgelasteten Workern — keine manuelle Konfiguration nötig.
- `current_load` ist bereits Teil des `Worker` -Proto-Messages; kein zusätzliches Protokoll erforderlich.
- Strategie bleibt zustandslos im Dispatcher (kein persistenter Zähler).
- Auswahl ist im Log als `strategy="least_loaded"` nachvollziehbar.

Nachteile:

- Wenn alle Worker `current_load=0` melden, entspricht das Verhalten faktisch einer Auswahl des ersten

Listenelements (abhängig von der Reihenfolge des Namensdienstes).

- Veralterte oder konstant falsch gemeldete `current_load` -Werte reduzieren die Effektivität.
- Keine Berücksichtigung von CPU- oder Speicherauslastung — nur Anzahl der Tasks.

ADR-004: Task- und Ergebnisspeicherung — In-Memory-Dictionary

Kontext

Der Dispatcher muss Tasks und ihre Ergebnisse persistent (innerhalb einer Laufzeit) speichern, damit Clients den Status und das Ergebnis eines Tasks jederzeit per `getResult` abfragen können. Es war zu entscheiden, ob ein externer Datenbankdienst, eine eingebettete Datenbank oder eine einfache In-Memory-Struktur eingesetzt wird.

Entscheidung

In-Memory-Dictionary (`TaskStore : dict[int, TaskRecord]`) mit Thread-Safety via `threading.Lock`. Tasks werden ausschließlich im Prozessspeicher des Dispatchers gehalten. Es gibt keine Persistenz über Prozessneustarts hinaus.

Alternativen

Alternative	Bewertung
Relationale Datenbank (PostgreSQL/SQLite)	ACID-Garantien, Persistenz, SQL-Abfragen — aber erheblicher Setup-Aufwand, externe Abhängigkeit, höhere Latenz für jeden Zustandsübergang.
Redis	Schneller In-Memory-Store mit optionaler Persistenz, gut für verteilte Systeme — aber zusätzlicher Dienst, Netzwerkkommunikation für jeden Lese-/Schreibzugriff.
Eingebettete DB (SQLite/ LMDB)	Persistenz ohne externen Dienst — aber Dateisystem-I/O bei jedem Zustandsübergang, Komplexität durch Transaktionsmanagement.
In-Memory-Dict (gewählt)	Minimal, keine Abhängigkeiten, O(1)-Zugriff, kein Netzwerk-Overhead — ausreichend für den Projektumfang.

Konsequenzen

Vorteile:

- Kein externer Dienst erforderlich — weniger Infrastruktur, einfacheres Deployment.
- O(1)-Zugriff per `task_id` für alle Zustandsübergänge.
- Thread-sicherer Zugriff durch `threading.Lock` ohne externe Koordination.
- Alle Daten liegen im Prozessspeicher — minimale Latenz für Dispatcher-interne Operationen.

Nachteile:

- **Keine Persistenz:** Bei einem Dispatcher-Neustart gehen alle Task-Zustände und Ergebnisse verloren.
- **Kein horizontales Scaling:** Mehrere Dispatcher-Instanzen würden separate, inkonsistente Stores haben.
- Speicherverbrauch wächst unbegrenzt mit der Anzahl der Tasks — für Langzeitbetrieb wäre ein Eviction-Mechanismus nötig.

Akzeptanz:

Diese Einschränkungen sind im Rahmen der Prüfungsaufgabe akzeptabel: Der Dispatcher ist als Single Instance konzipiert, und Task-Verlust bei Neustart ist kein funktionales Anforderungskriterium.

ADR-005: Heartbeat-Mechanismus — Intervall und Timeout-Schwellwerte

Kontext

Worker müssen dem Namensdienst periodisch signalisieren, dass sie noch verfügbar sind. Fällt ein Worker aus, soll der Namensdienst ihn nicht länger als aktiv melden, damit der Dispatcher keine Tasks an einen nicht erreichbaren Worker sendet. Es war zu entscheiden, wie häufig Heartbeats gesendet werden und wann ein Worker als inaktiv gilt.

Entscheidung

Folgende Standardwerte werden eingesetzt (alle via Umgebungsvariable konfigurierbar):

Parameter	Wert	Konfiguration
Heartbeat-Intervall (Worker)	10 Sekunden	HEARTBEAT_INTERVAL_SEC
UNHEALTHY-Schwelle (Namensdienst)	30 Sekunden	HEARTBEAT_TIMEOUT_SEC
OFFLINE-Schwelle (Namensdienst)	90 Sekunden	OFFLINE_TIMEOUT_SEC

Der Namensdienst stuft Worker in zwei Stufen herab:

1. Kein Heartbeat für ≥ 30 s $\rightarrow$ `ACTIVE` $\rightarrow$ `UNHEALTHY` (nicht mehr für Dispatch verfügbar)
2. Kein Heartbeat für ≥ 90 s $\rightarrow$ `UNHEALTHY` $\rightarrow$ `OFFLINE` (als ausgefallen markiert)

Worker im Zustand `DRAINING` (aktiver Shutdown) werden vom Health-Checker nicht angetastet.

Alternativen

Ansatz	Bewertung
Sehr kurzes Intervall (1–2 s)	Schnelle Ausfallentdeckung — aber hohes Netzwerkaufkommen bei vielen Workern, erhöhte Last auf dem Namensdienst.
Sehr langes Intervall (60+ s)	Wenig Netzwerklast — aber Tasks werden bis zu 60 s lang an einen toten Worker gesendet, bevor der Dispatcher einen Timeout bemerkt.

Ansatz	Bewertung
10 s Intervall / 30 s Timeout (gewählt)	Ausgewogenes Verhältnis: drei ausbleibende Heartbeats lösen einen Statuswechsel aus — toleriert kurzfristige Netzwerkunterbrechungen, ohne sofort false positives zu produzieren.
Push-basiert (Worker meldet Shutdown)	Sofortige Abmeldung bei geordnetem Shutdown — aber kein Schutz vor ungeplanten Abstürzen. Ergänzend implementiert via <code>DeregisterWorker</code> bei SIGTERM.

Konsequenzen

Vorteile:

- Drei aufeinanderfolgende ausbleibende Heartbeats lösen erst dann `UNHEALTHY` aus — transiente Netzwerkfehler erzeugen keine false positives.
- Die zweistufige Herabstufung (`UNHEALTHY` → `OFFLINE`) ermöglicht zukünftige Unterscheidung zwischen „kurz nicht erreichbar“ und „definitiv ausgefallen“.
- Alle Schwellwerte sind via Umgebungsvariablen anpassbar — kein Code-Change für andere Deployment-Umgebungen nötig.
- Worker melden sich zusätzlich aktiv ab (`DeregisterWorker` bei SIGTERM), wodurch geordnete Shutdowns sofort erkannt werden.

Nachteile:

- Im ungünstigsten Fall dauert es bis zu 30 s, bis ein ausgefallener Worker als `UNHEALTHY` gilt; in dieser Zeit kann der Dispatcher Tasks an ihn senden, die dann durch den Timeout-Mechanismus erneut eingeplant werden müssen.
- Der Health-Checker-Thread im Namensdienst läuft sekundlich — bei sehr vielen Workern (hunderte) könnte dies zu Lock-Contention führen.

ADR-006: Retry-Strategie bei Task-Timeout — Re-Enqueue mit Worker-Wechsel

Kontext

Der Dispatcher muss erkennen, wenn ein Worker nicht innerhalb eines definierten Timeouts antwortet. Es war zu entscheiden, wie mit dem betroffenen Task umgegangen wird: sofortiges Fehlschlagen, Re-Dispatch an denselben Worker oder Re-Enqueue mit Worker-Wechsel.

Entscheidung

Begrenzte Re-Enqueue-Strategie mit Worker-Wechsel-Präferenz:

- Timeout-Checker prüft alle 5 Sekunden Tasks im Zustand `DISPATCHED` / `PROCESSING` .
- Bei Timeout: `TIMEOUT` → `RETRYING` → `QUEUED` (wenn `retry_count` < `MAX_RETRIES`).
- Bei Retry wird der zuletzt fehlgeschlagene Worker ausgeschlossen (`last_failed_worker`).

- Bei `retry_count >= MAX_RETRIES` : direkt `FAILED` .

Konfiguration: `TASK_TIMEOUT_SEC` (Standard: 60), `MAX_RETRIES` (Standard: 3).

Alternativen

Strategie	Bewertung
Kein Retry — sofort FAILED	Einfachste Implementierung — unrobust bei transienten Worker-Ausfällen.
Direkter Re-Dispatch (ohne Re-Enqueue)	Schneller — aber umgeht die Queue-Priorität und erhöht Dispatcher-Komplexität.
Unbegrenzte Retries	Kein Task-Verlust — Endlosschleife bei dauerhaft fehlerhaften Payloads möglich.
Exponential Backoff vor Re-Enqueue	Reduziert Last bei vielen gleichzeitigen Timeouts — erhöht aber Komplexität und Wartezeit.
Re-Enqueue mit Worker-Wechsel (gewählt)	Robust gegenüber transienten Ausfällen, begrenzt durch <code>MAX_RETRIES</code> , vollständig in Logs nachvollziehbar.

Konsequenzen

Vorteile:

- Tasks überleben transiente Worker-Abstürze automatisch.
- Zustandshistorie (`TIMEOUT` → `RETRYING` → `QUEUED`) vollständig in strukturierten Logs nachvollziehbar.
- `MAX_RETRIES=0` deaktiviert Retries ohne Code-Änderung.
- Worker-Wechsel-Präferenz verhindert, dass ein defekter Worker wiederholt denselben Task erhält.

Nachteile:

- Maximale Gesamtverarbeitungszeit eines Tasks: $(\text{MAX_RETRIES} + 1) \times \text{TASK_TIMEOUT_SEC}$.
- Tasks mit fehlerhafte Payloads (die jeden Worker zum Absturz bringen) scheitern erst nach `MAX_RETRIES` Versuchen.